

RESUMEN DESARROLLO DE SOFTWARE

Resumen 1-3

(1) ARQUITECTURA DE UN SISTEMA WEB MODERNO

La mayoría de los sistemas web actuales se dividen en tres componentes principales que trabajan de forma conjunta:

- **Frontend (Capa de Presentación):** Interfaz con la que interactúa el usuario en el navegador o aplicación móvil. Se encarga de mostrar información, gestionar la interacción y enviar solicitudes al backend.
- **Backend (Capa de Lógica):** Se ejecuta en el servidor y procesa las solicitudes del frontend. Es donde residen las **reglas de negocio**, la gestión de autenticación/autorización y la validación de datos.
- **Base de Datos (Capa de Persistencia):** Almacena la información de manera duradera (usuarios, pedidos, productos).

En sistemas más grandes pueden aparecer capas adicionales (caché, colas de mensajes, servicios de archivos, etc.)

Beneficios de esta separación: facilita la organización del sistema (cada capa tiene una frontera clara y se comunica con las demás a través de interfaces bien definidos), facilita el mantenimiento (puedes cambiar una capa sin reescribir todo), mejora la escalabilidad (puedes agregar más servidores de backend si es necesario) y permite el trabajo en equipo distribuido.

Cada una de estas capas cumple un rol diferente dentro del sistema. La comunicación entre capas suele ser unidireccional en el flujo de una solicitud; las respuestas vuelven en sentido inverso.

Usuario → frontend → backend (API) → bases de datos

FRONTEND

Parte del sistema con la que interactúan directamente los usuarios. Se ejecuta en el cliente: navegador (aplicaciones web) o dentro de una aplicación nativa o híbrida (móvil o escritorio). Se encarga de mostrar la información en pantalla y permitir la interacción con la aplicación.

Responsabilidades

- Mostrar la interfaz de usuario (UI) y asegurar que sea usable en distintos dispositivos y tamaños de pantalla (diseño responsive).
- Gestionar la interacción con el usuario y dar feedback inmediato (estados de carga, mensajes de error, animaciones).
- Enviar solicitudes al backend (por ejemplo vía HTTP a una API REST) y manejar las respuestas.
- Mostrar datos recibidos desde el backend y mantener la interfaz actualizada (estado local o global según la complejidad de la app).
- Realizar validaciones básicas en el cliente (campos obligatorios, formato de email, etc.) para mejorar la experiencia; las validaciones que afectan seguridad o reglas de negocio deben repetirse siempre en el backend.

No debería contener la lógica de negocio principal del sistema. Esa lógica pertenece al backend porque: (1) el código del frontend es visible y modificable por el usuario, por lo que no se puede confiar en él para decisiones críticas; (2) mantener la lógica en un solo lugar (backend) evita inconsistencias entre web, móvil y otros clientes.

El frontend web se descarga desde un servidor (archivos estáticos o generados) y se ejecuta en el navegador. 2 enfoques (muchas apps combinan ambas):

- **Aplicación de una sola página (SPA):** la aplicación se carga una vez; las “páginas” son cambios de vista manejados por JavaScript sin recargar el documento. El backend expone una API (por ejemplo, REST) y el frontend consume esos datos. Bueno para interfaces muy interactivas; el primer contenido visible puede demorar más si la app es pesada.
- **Renderizado en servidor (SSR) o multipágina:** cada navegación puede solicitar al servidor una nueva página HTML ya generada. El servidor puede consultar datos y armar el HTML antes de enviarlo. Mejor para SEO y para que el contenido visible aparezca rápido; la interacción posterior puede seguir usando JavaScript.

Tecnologías comunes:

HTML — estructura del contenido (semántica, accesibilidad).

CSS — estilos visuales, layout (flexbox, grid), diseño responsive, variables (custom properties).

JavaScript — comportamiento dinámico, llamadas a la API, manejo del estado de la interfaz.

Proyectos modernos suelen usar frameworks o bibliotecas que organizan el código y facilitan componentes reutilizables, enrutamiento y gestión de estado:

- React (biblioteca de componentes; ecosistema amplio).
- Angular (framework completo con estructura y herramientas integradas).
- Vue.js (framework progresivo, fácil de integrar en proyectos existentes).

Habitual usar herramientas de build (bundlers como Webpack, Vite o esbuild) que transforman el código fuente, gestionan dependencias y generan los archivos estáticos que se sirven al navegador.

BACKEND

se ejecuta en el servidor (en la nube, en un datacenter o en un contenedor) y se encarga de procesar las solicitudes provenientes del frontend u otros clientes. Donde se concentran las reglas de negocio y el acceso a los datos; el frontend solo presenta la información y envía acciones.

Responsabilidades

- Implementar la lógica de negocio: reglas que definen cómo funciona el sistema (cálculos, validaciones de dominio, flujos de trabajo). Esta lógica debe vivir en el servidor para garantizar consistencia y seguridad.
- Gestionar autenticación y autorización: verificar la identidad del usuario (login, tokens) y decidir qué operaciones puede realizar (permisos por rol o recurso).
- Validar todos los datos recibidos del cliente; no se debe confiar en que el frontend envíe datos correctos o que no intenten saltarse restricciones.
- Comunicarse con la base de datos para leer y persistir información, manteniendo integridad y, cuando corresponda, transacciones.

- Integrarse con otros servicios o APIs externas (pagos, envío de emails, almacenamiento de archivos, etc.).
- Manejar errores de forma controlada y registrar logs del sistema para diagnóstico, auditoría y monitoreo

Organización interna

- Capa de presentación o API (controllers, handlers): recibe las peticiones HTTP, extrae parámetros y body, y delega en la capa de aplicación. Devuelve las respuestas (JSON, códigos de estado).
- Capa de aplicación o servicios: contiene la lógica de negocio orquestando use cases (por ejemplo "crear pedido", "procesar pago"). No conoce los detalles de HTTP ni de la base de datos.
- Capa de acceso a datos (repositories, DAOs): encapsula las consultas y escrituras a la base de datos. El resto del backend trabaja con entidades o modelos de dominio, no con SQL crudo.

Así se facilita el mantenimiento, las pruebas unitarias y el cambio de infraestructura (por ejemplo, cambiar de base de datos tocando solo la capa de acceso).

Escalabilidad y observabilidad

backend suele diseñarse sin estado (stateless) en cada instancia: no se guarda sesión en memoria en el servidor, de modo que cualquier instancia pueda atender cualquier solicitud. Permite escalar horizontalmente (agregar más servidores detrás de un balanceador de carga).

Para operar el sistema en producción es importante la observabilidad: logs estructurados, métricas (tiempos de respuesta, cantidad de errores) y, en sistemas complejos, trazabilidad distribuida para seguir una solicitud a través de varios servicios.

Tecnologías comunes

- Node.js (JavaScript o TypeScript en el servidor; ecosistema npm, adecuado para I/O intensivo).
- Java + Spring Boot (muy usado en entornos empresariales, gran ecosistema y rendimiento).
- Python (Django, Flask, FastAPI): rápido para prototipar y para APIs; muy usado en datos y ML).
- .NET (C#) (ecosistema Microsoft, multiplataforma con .NET Core).
- Go (Golang) (compilado, simple, muy usado en servicios de infraestructura y microservicios).

BASES DE DATOS

Almacena la información del sistema de manera persistente: los datos sobreviven a reinicios del servidor y deben conservar integridad y consistencia. En operaciones que afectan varios registros de forma coordinada, el backend usa **transacciones**: un conjunto de operaciones que se ejecutan como una unidad; si algo falla, se revierte todo (rollback). Las bases relacionales ofrecen propiedades **ACID** (Atomicidad, Consistencia, Aislamiento, Durabilidad).

Tipos

- **Relacionales (SQL)**: información en tablas con columnas definidas y relaciones entre tablas (claves foráneas). Opción habitual cuando los datos tienen estructura estable y se necesitan consultas complejas e integridad referencial.
- **NoSQL**: modelos más flexibles:
 - *Documentos* (MongoDB): almacenan documentos JSON con esquema flexible.

- *Clave-valor* (Redis): muy rápidas para leer/escribir por clave. Se usan como caché o para sesiones.
- *Columnar* (Cassandra): optimizadas para grandes volúmenes y escritura masiva.

La elección entre SQL y NoSQL depende del modelo de datos. En muchos sistemas se usan ambos: una base relacional como fuente de verdad y Redis u otra para caché o sesiones.

SISTEMAS DISTRIBUIDOS

En aplicaciones simples, todo puede correr en una misma máquina. En la mayoría de los sistemas modernos cada componente corre en servidores distintos, a veces en distintas ubicaciones geográficas.

Permite:

- Escalabilidad: escalar cada parte según su necesidad. No hace falta duplicar todo el stack en una sola máquina gigante.
- Disponibilidad: si un servidor de aplicaciones cae, otros pueden seguir atendiendo; la base de datos puede tener réplicas para evitar un único punto de fallo. El frontend estático puede servirse desde una CDN con alta disponibilidad.
- Distribución de la carga: el trabajo se reparte entre servicios; cada uno puede optimizarse (por ejemplo, servidores con mucha RAM para la base de datos, muchos núcleos para el backend).
- Aislamiento de fallos: un problema en un componente (por ejemplo, saturación del backend) no tiene por qué tumbar la base de datos ni el servidor de archivos estáticos si están separados.

FLUJO TIPICO DE UNA SOLICITUD EN UN SISTEMA WEB

Usuario → frontend → request HTTP → backend → consulta → base de datos → respuesta → backend → frontend → interfaz actualizada

CACHE EN EL FLUJO

En sistemas reales suele existir caché en uno o más puntos para reducir carga y mejorar tiempos de respuesta:

- Navegador: puede cachear recursos estáticos (HTML, CSS, JS, imágenes) y a veces respuestas de la API según headers (Cache Control, ETag).
- CDN: puede servir el frontend estático y, en algunos casos, respuestas de la API para contenido poco cambiante.
- Backend: puede usar una capa de caché (ej: Redis) para no consultar la base de datos en cada request cuando los datos no cambian a cada segundo (ej: listados de categorías, datos de sesión).

Así se reduce la cantidad de consultas a la base de datos y la latencia percibida por el usuario.

Cuando algo falla se debe contemplar:

- Timeouts: el frontend y el backend no deben esperar indefinidamente; tras un tiempo razonable se considera fallida la solicitud.
- Códigos de error HTTP y mensajes claros para que el frontend pueda mostrar un mensaje al usuario o reintentar.
- Reintentos (con cuidado en operaciones que no son idempotentes) y circuit breakers en sistemas más avanzados, para no saturar un servicio que ya está fallando.

ARQUITECTURA CLIENTE-SERVIDOR

Un componente actúa como solicitante (cliente) y otro como proveedor de servicios (servidor).

Es la base sobre la que se apoyan HTTP, las APIs REST, el despliegue de frontends y backends y la forma en que se escalan las aplicaciones (múltiples clientes conectados a uno o más servidores). Permite comparar con otros modelos a la hora de diseñar un sistema.

- Cliente: inicia la comunicación y solicita recursos o operaciones.
- Servidor: espera solicitudes, las procesa y devuelve respuestas.

Se comunican a través de una red, generalmente utilizando protocolos como HTTP sobre TCP/IP.

Concepto general

Un componente actúa como solicitante (cliente) y otro como proveedor de servicios (servidor). La comunicación sigue un patrón **request-response**: el cliente inicia siempre la comunicación, el servidor espera solicitudes (escuchando en un puerto), las procesa y devuelve una respuesta. El servidor no envía datos por iniciativa propia en este modelo básico.

Cliente → request → servidor → procesamiento → response → cliente

Asimetría: muchos clientes, uno o más servidores que centralizan la lógica y los datos.

CLIENTE

El cliente es el componente que interactúa directamente con el usuario y que envía solicitudes al servidor para obtener información o realizar operaciones. No necesita conocer la implementación interna del servidor; solo debe saber cómo comunicarse con él (protocolo, formato de mensajes, endpoints). Permite que un mismo servidor sea usado por distintos tipos de clientes (web, móvil, otro sistema) sin cambiar la lógica del servidor.

Responsabilidades

- Mostrar la interfaz de usuario y gestionar la experiencia (navegación, formularios, estados de carga).
- Recibir acciones del usuario y traducirlas en solicitudes al servidor ("guardar" → POST /pedidos).
- Enviar solicitudes al servidor con los parámetros o el body adecuados.
- Mostrar los datos recibidos en la respuesta y manejar errores (mensajes, reintentos) cuando la comunicación falla.

Se suele hablar de cliente liviano (thin client) cuando la lógica y los datos están mayormente en el servidor y el cliente solo presenta la interfaz, y de cliente pesado (thick client) cuando el cliente tiene más lógica y tal vez caché local; en la web actual lo habitual es un cliente liviano que consume una API.

SERVIDOR

Componente encargado de recibir las solicitudes enviadas por los clientes, procesarlas y devolver una respuesta. Suele estar en ejecución de forma continua (o escalado mediante varias instancias), escuchando en un puerto de red a la espera de conexiones. Un mismo servidor (o un conjunto de instancias detrás de un balanceador) atiende a múltiples clientes de forma concurrente; cada solicitud se maneja de forma independiente, y en un diseño stateless el servidor no "recuerda" al cliente entre una petición y la siguiente salvo que la información venga en la propia petición (ej: un token).

Se encarga de:

- Procesar la lógica de negocio: reglas, cálculos y flujos que definen el comportamiento del sistema.
- Validar todos los datos enviados por el cliente; no se confía en que el cliente envíe datos correctos o seguros.
- Consultar o modificar información en la base de datos y garantizar consistencia cuando corresponda (transacciones).
- Gestionar autenticación y autorización: quién es el usuario y qué puede hacer.
- Integrarse con otros servicios o APIs externas (pagos, envío de correo, almacenamiento, etc.).

En una aplicación web, el servidor corresponde a la capa backend. Puede estar desplegado en una sola máquina o en varias (escalado horizontal) detrás de un balanceador de carga que reparte las peticiones entre instancias.

COMUNICACION CLIENTE-SERVIDOR

Ocurre a través de una red usando protocolos que definen el formato de los mensajes y las reglas del intercambio. El protocolo actúa como un contrato: si ambos lados lo respetan, pueden interoperar aunque estén implementados en tecnologías distintas.

El protocolo más utilizado es **HTTP**, que se apoya sobre **TCP/IP**: primero se establece una conexión TCP (con handshake), y sobre esa conexión se envían los mensajes HTTP. En HTTP/1.1 es habitual reutilizar la misma conexión para varias solicitudes (**keep-alive**) para reducir la sobrecarga.

Cada solicitud incluye un método (GET, POST, etc.), una URL, headers y opcionalmente un body; la respuesta incluye un código de estado, headers y un body. Los datos suelen enviarse en formato **JSON** (header `Content-Type: application/json`).

Para comunicación cifrada se usa **HTTPS**: HTTP sobre TLS/SSL, los mensajes viajan cifrados y el cliente puede verificar la identidad del servidor mediante certificados

Ventajas:

- **Separación de responsabilidades:** dividir el sistema en componentes con funciones específicas: el cliente → interfaz y experiencia del usuario; el servidor → lógica de negocio y los datos. Facilita el mantenimiento (cambios localizados), la evolución y el trabajo en equipo (equipos distintos pueden encargarse de cliente y servidor).
- **Centralización de la lógica y los datos:** La lógica crítica y los datos permanecen en el servidor. mejorando la seguridad (el cliente no tiene acceso directo a la base de datos ni a secretos), la integridad (reglas y validaciones en un solo lugar) y la consistencia entre distintos clientes que consumen el mismo backend.
- **Escalabilidad:** Se puede escalar el servidor (+ CPU, + memoria, o + instancias detrás de un balanceador) para atender más clientes o más carga, sin que cada cliente tenga que ser más potente. Los clientes suelen ser numerosos y dispersos; centralizar la capacidad en el servidor permite optimizar costos y rendimiento.
- **Reutilización y heterogeneidad:** Un mismo servidor (API) puede ser utilizado por distintos clientes. Se pueden usar tecnologías distintas en cliente y servidor (ej: frontend en JavaScript, backend en Java), siempre que ambos respeten el protocolo y el formato de los mensajes.

Desventajas:

- **Dependencia del servidor:** Si el servidor falla o deja de estar disponible, los clientes no pueden acceder al sistema o completar operaciones. Mitigaciones: redundancia (varias instancias del servidor detrás de un balanceador), monitoreo y recuperación automática, y en el cliente manejar errores y mensajes claros cuando el servidor no responde (timeout, 503, etc.).
- **Latencia de red:** La comunicación depende de la red: distancia, congestión y calidad del enlace generan demoras (latencia) y posibles pérdidas de paquetes. Mitigaciones: caché en el cliente o en capas intermedias (CDN), diseño de APIs que minimicen round-trips, y para casos que requieran baja latencia o tiempo real, valorar complementos como WebSockets o conexiones persistentes.
- **Carga en el servidor:** Si muchos clientes realizan solicitudes al mismo tiempo, el servidor puede saturarse (CPU, memoria, conexiones a la base de datos) si no está correctamente dimensionado o escalado. Mitigaciones: escalado horizontal (más instancias), caché (reducir trabajo repetido), rate limiting para evitar abusos, y optimización de consultas y recursos.
- **Seguridad y confiabilidad del cliente:** El código y los datos que maneja el cliente están en un entorno que el usuario o un atacante puede inspeccionar o modificar. Por eso nunca se debe confiar en el cliente para decisiones de seguridad ni para guardar secretos; el servidor debe validar y autorizar todas las operaciones. Usar HTTPS en todo el tráfico para evitar interceptación y suplantación.

usuario → navegador (usuario) → request HTTP → servidor → consulta → BD → responde HTTP → navegador → interfaz actualizada

PROTOCOLO HTTP

El HTTP (Hypertext Transfer Protocol) permite que clientes y servidores intercambien información a través de una red (generalmente Internet) mediante mensajes de solicitud y respuesta con un formato bien definido. Cada vez que un usuario accede a una página web, envía un formulario o una aplicación solicita datos a una API, se produce comunicación basada en HTTP.

HTTP es un protocolo de la capa de aplicación del modelo TCP/IP: se apoya en TCP para el transporte fiable de los datos. El protocolo define cómo deben estructurarse las solicitudes y respuestas (método, URL, headers, body, códigos de estado) para que cualquier cliente y cualquier servidor que lo implementen puedan interoperar.

Protocolo

Conjunto de reglas que define cómo deben comunicarse dos o más sistemas. Sin un protocolo acordado, cada implementación podría enviar y esperar mensajes de forma distinta y no habría interoperabilidad.

Establece aspectos como:

- cómo se envían los mensajes (orden, secuencia)
- cómo se interpretan (sintaxis y semántica de los campos)
- qué formato deben tener (por ejemplo, línea de inicio, headers, body)
- qué tipo de respuestas o códigos pueden generarse y qué significan

Los protocolos suelen organizarse en capas (stack de protocolos): HTTP usa TCP para el transporte, TCP usa IP para el enrutamiento, etc. Así cada capa tiene una responsabilidad clara y se pueden combinar protocolos de distintas capas.

- HTTP (Hypertext Transfer Protocol): Define el formato de solicitudes y respuestas y los métodos (GET, POST, etc.). Es la base de las APIs REST y de la navegación web.
- HTTPS (HTTP Secure): HTTP sobre TLS/SSL: los mensajes viajan cifrados y el cliente puede verificar la identidad del servidor mediante certificados. Recomendada en producción para proteger datos y evitar suplantación.
- HTTP/2 y HTTP/3: Evoluciones del protocolo: HTTP/2 permite multiplexación de peticiones sobre una misma conexión y compresión de headers; HTTP/3 usa QUIC (sobre UDP) en lugar de TCP para reducir latencia en redes inestables. La semántica de métodos, códigos de estado y headers se mantiene.
- FTP (File Transfer Protocol): Transferencia de archivos entre sistemas (subida y descarga). Menos usado en la web actual; muchas veces se reemplaza por HTTP(S) o por APIs de almacenamiento.
- SMTP (Simple Mail Transfer Protocol): Envío de correo electrónico entre servidores de correo.
- TCP/IP: Conjunto de protocolos de las capas de transporte (TCP) y red (IP). TCP garantiza entrega ordenada y sin pérdidas; IP se encarga del enrutamiento.
- WebSocket: Protocolo de capa de aplicación que permite comunicación bidireccional y persistente sobre una sola conexión. Útil para tiempo real (chat, notificaciones). Se inicia con una handshake HTTP y luego se "eleva" a WebSocket.

FUNCIONAMIENTO HTTP

1. conexión TCP con el servidor (handshake) → sobre esa conexión se envían los bytes (request)
2. cliente envía solicitud
3. servidor devuelve una sola respuesta asociada

En HTTP/1.1 se mantiene la conexión abierta para varias solicitudes (keep-alive), evitando el costo de abrir y cerrar TCP para cada petición.

Estructura de una request HTTP

formada por una línea de inicio (request line), headers (cabeceras) y, opcionalmente, un body (cuerpo).

- Línea de inicio: Incluye el método HTTP, la ruta (path) y la versión del protocolo (ej: HTTP/1.1). La ruta puede incluir query string después de ? (ej: /productos?categoria=tech). El host (dominio) suele ir en el header Host, ya que una misma IP puede alojar varios sitios.
- Headers: valor que aportan contexto y metadatos.
 - Host: dominio del servidor (obligatorio en HTTP/1.1).
 - Content-Type: tipo del body (por ejemplo application/json, application/x-www-form-urlencoded).
 - Content-Length: tamaño del body en bytes (cuando hay body).
 - Authorization: credenciales o token (por ejemplo Bearer <token>).
 - Accept: tipos de respuesta que el cliente acepta (por ejemplo application/json).
 - User-Agent: identificación del cliente (navegador, app); útil para estadísticas y compatibilidad.

- Cookie: cookies enviadas al servidor para mantener estado (sesión, preferencias).
- Body (opcional): En métodos como POST o PUT suele ir el cuerpo de la petición (datos del formulario, JSON de la API). En GET normalmente no se usa body; los parámetros van en la URL (query string).

Estructura response HTTP

respuesta del servidor consta de una línea de estado (status line), headers y, en la mayoría de los casos, un body.

- Línea de estado: Incluye la versión del protocolo, el código de estado numérico (200, 404, 500, etc.) y una frase descriptiva (OK, Not Found, Internal Server Error). El código es lo que usa el cliente para decidir cómo actuar.
- Headers: Metadatos de la respuesta
 - Content-Type: tipo del body (por ejemplo application/json, text/html). El cliente lo usa para interpretar el contenido
 - Content-Length: tamaño del body en bytes.
 - Cache-Control: indica si y cómo se puede cachear la respuesta (p. ej. max-age=3600, no-cache).
 - Set-Cookie: envía una cookie al cliente para que la envíe en futuras peticiones (sesión, preferencias).
 - Access-Control-Allow-Origin (y otros CORS): indican si el origen del cliente puede leer la respuesta; necesario cuando el frontend está en otro dominio.
 - ETag, Last-Modified: permiten peticiones condicionales (el cliente puede pedir "solo si cambió" y recibir 304 Not Modified sin body).
- Body: Contenido de la respuesta (HTML de la página, JSON de la API, etc.). En algunas respuestas no hay body: por ejemplo 204 No Content o 304 Not Modified. En APIs modernas el body suele ser JSON cuando la respuesta lleva datos estructurados.

Códigos de estado HTTP

indican el resultado de la solicitud de forma estandarizada.

- 1xx (Informativos): la petición se recibió y el proceso continúa
- 2xx (Éxito): la solicitud se entendió y se procesó correctamente. 200 OK; 201 created; 204 no content (éxito, pero sin cuerpo en la respuesta)
- 3xx (Redirección): el cliente debe hacer una acción adicional (normalmente seguir otra URL). 301 moved permanently; 302 found; 304 not modified.
- 4xx (Error del cliente): la solicitud está mal formada, no autorizada o el recurso no existe; el cliente debe corregir algo. 400 bad request; 401 unauthorized; 403 forbidden; 404 not found.
- 5xx (Error del servidor): el servidor falló al procesar una solicitud válida; el cliente puede reintentar más tarde. 500 internal server error; 502 bad gateway; 503 service unavailable.

Usuario → navegador → request → servidor → consulta BD → response → navegador → interfaz actualizada

Herramientas para inspeccionar y probar HTTP

- DevTools del navegador: Permiten ver todas las peticiones HTTP que hace la página. Es la forma más directa de comprobar qué envía el frontend y qué devuelve el servidor, y de detectar errores 4xx/5xx o respuestas lentas.
- Curl: Herramienta de línea de comandos para enviar peticiones HTTP. Sirve para probar una API desde la terminal, automatizar scripts o reproducir una request sin abrir el navegador.
- Postman, insomnia, etc: Clientes gráficos para diseñar, guardar y ejecutar colecciones de peticiones HTTP. Permiten definir variables, entornos (desarrollo, producción) y automatizar pruebas de APIs. Facilita el debugging, la integración con backends y la documentación de las APIs que se desarrollan o consumen.

API REST

Define qué operaciones se pueden realizar y cómo deben realizarse. Funciona como un contrato entre quien expone el servicio (el servidor) y quien lo consume (el cliente). Para que esta comunicación sea posible, las APIs suelen estar documentadas: se especifican los endpoints, los parámetros, los formatos de entrada y salida, y los códigos de error. Herramientas como OpenAPI (Swagger) permiten describir una API de forma estándar y generar documentación interactiva o clientes automáticos.

Tipos según alcance:

- APIs internas: utilizadas dentro de una misma organización (ej: el frontend consumiendo al backend de la misma aplicación).
- APIs públicas: expuestas a terceros para integrar servicios (ej: APIs de Google Maps, pasarelas de pago, redes sociales).
- APIs de partners: compartidas con socios o clientes específicos bajo acuerdos y autenticación controlada.

En aplicaciones web, las APIs suelen funcionar sobre HTTP.

Cliente → request → API → procesamiento → response → cliente (front web, app móvil, sistema o microservicio, servicio externo)

Principios de REST

- Recursos identificables: la información se organiza en recursos que se identifica de forma única mediante un URI.
- Stateless (sin estado en el servidor).
- Uso de métodos HTTP con significado
- Respuestas con códigos de estado estándar.

Autenticación y autorización en APIs

APIs no públicas requieren que el cliente demuestre quién es (autenticación) y que el servidor verifique qué puede hacer (autorización). Como REST es stateless, esa información no se guarda en sesión en el servidor; debe ir en cada solicitud.

- API Key: un valor fijo (ej: en un header o en query) que identifica al cliente. Si se filtra cualquiera puede suplantar al cliente.
- Bearer token (JWT): tras un login, el servidor devuelve un token (puede incluir identidad, permisos y tener caducidad) que el cliente envía en el header.

- OAuth 2.0: estándar para delegar acceso ("acceder con Google").

Otras formas de comunicación entre sistemas

- **GraphQL:** El cliente especifica exactamente qué datos necesita en una sola petición, evitando traer datos de más (*over-fetching*) o necesitar múltiples llamadas (*under-fetching*). Expone un esquema y uno o pocos endpoints en lugar de muchos como REST. → *flexibilidad en los datos*.
- **WebSockets:** Conexión bidireccional y persistente: el servidor puede enviar datos sin que el cliente los solicite. No reemplaza a REST para el flujo clásico petición-respuesta. → *tiempo real*.
- **gRPC:** Protocolo sobre HTTP/2 con definiciones de servicio. Alta eficiencia en ancho de banda y latencia. → *microservicios y comunicación servidor a servidor*.
- **Sistemas de mensajería (colas y streams):** Comunicación asíncrona: el productor envía mensajes a una cola o topic y el consumidor los procesa cuando puede, sin respuesta HTTP inmediata. → *desacoplamiento y procesamiento diferido*.

(3) PATRON MVC Y DISEÑO DE APIs REST

Separation of CAoncerns (SoC)

Cada parte de un sistema debe ocuparse de un único tipo de problema.

Single Responsibility Principle (SRP)

cada módulo, clase o función debe tener una única razón para cambiar. Si un módulo puede verse afectado por cambios en la lógica del negocio y también por cambios en el formato de presentación, entonces ese modulo tiene dos responsabilidades y viola SRP.

Cuando un sistema se construye sin respetar esto → "código espagueti": un conjunto de funciones y módulos tan entrelazados que modificar cualquier parte resulta impredecible. Pruebas imposibles porque no hay manera de testear una parte sin activar otras. Incorporar nuevos desarrolladores consume semanas porque el código no tiene una estructura explicable.

PATRON MVC (Model-View-Controller)

Dividir una aplicación en tres componentes con responsabilidades distintas e independientes. Cada componente puede cambiar sin afectar a los otros, siempre que respete los contratos que los unen.

- **Model:** representación de los datos del dominio. Define que información existe en el sistema, como está estructurada y que reglas deben cumplirse para que esa información sea válida. Contiene la lógica de validación: las reglas que determinan si un dato es aceptable.

Un error frecuente es delegar las validaciones al Controller o al frontend.

- **View:** capa de presentación: lo que el usuario ve y con lo que interactúa. Muestra la información que proviene del Model de una manera comprensible y visualmente coherente. Responsabilidad de la View es exclusivamente presentacional. No debe contener lógica de negocio, ni reglas de validación, ni acceso a datos.
- **Controller:** componente que conecta la View con el Model. Cuando el usuario realiza una acción, recibe esa acción, coordina las validaciones con el Model, actualiza los datos si corresponde y determina que debe mostrarse a continuación en la View. Contiene la lógica de la aplicación (ej: (que pasa cuando el usuario hace clic en "Eliminar")), que es diferente de la lógica del negocio.

Componente	Responsabilidad	Riesgo si se malinterpreta
Model	Datos, estructura, validaciones y persistencia.	Convertirlo en un simple contenedor de datos sin reglas.
View	Presentación visual e interacción con el usuario.	Incluir lógica de negocio o validaciones en la interfaz.
Controller	Coordinación entre View y Model; lógica de aplicación.	Convertirlo en el lugar donde vive toda la lógica del sistema.

MVC EN EL MUNDO REAL

- **Router:** recibe las solicitudes HTTP entrantes y las dirige al handler correcto según la URL y el método. No contiene lógica de negocio.
- **Handler (Controller):** recibe la solicitud del Router, extrae los datos del body o parámetros, invoca las validaciones del Model, llama al Store y construye la respuesta. Un handler bien diseñado es delgado: delega, no implementa lógica propia.
- **Model:** define la estructura de los datos y las reglas que deben cumplir. No sabe nada de HTTP ni de cómo se muestra la respuesta.
- **Store (Repository):** módulo responsable de leer y escribir datos en el sistema de persistencia. Traduce operaciones lógicas a operaciones concretas de base de datos. Si se cambia de motor de BD, el cambio está contenido en el Store y no contamina el resto del sistema.
- **Middleware:** funciones que se ejecutan en la cadena de procesamiento de cada request antes o después del handler. Implementan comportamientos que aplican a todos los requests (cross-cutting concerns): logging, autenticación, autorización.

Flujo de un request en el backend

Router identifica ruta y método → Handler extrae datos → valida contra el Model → si falla: responde 400 Bad Request → si es válido: invoca al Store → construye respuesta con código HTTP apropiado

MVC EN EL FRONTEND REAL

En React, los tres componentes de MVC aparecen distribuidos en el State, los Components y la capa de coordinación que vive en un componente raíz o en hooks personalizados.

- **State, components y coordinacion:** representación en memoria de los datos que el frontend necesita para funcionar; es el Model del frontend. Los Components son las piezas visuales: la View del frontend. Cada componente renderiza una porción de la pantalla y captura las acciones del usuario. La coordinación, equivalente al Controller, vive en un componente raíz o en hooks: es quien maneja el state, coordina las llamadas a la API y decide que mostrar según el estado del sistema.
- **Componentes inteligentes y tontos:** Un componente inteligente maneja state, coordina llamadas a la API y pasa datos a los componentes tontos como props. Un componente tonto solo recibe props y los renderiza; no tiene state propio, no hace llamadas a servicios externos y no toma decisiones sobre el flujo.
- **Capa API:** Toda comunicación del frontend con el backend debe pasar por una capa centralizada de funciones de acceso a la API. Esta capa contiene funciones. Ningún componente debería hacer solicitudes HTTP directamente. Si la URL del backend cambia, si el formato de la solicitud se modifica, o si se necesita agregar autenticación a todos los requests, esos cambios se hacen en un único lugar y el resto del sistema no se ve afectado.

REST (Representational State Transfer) API DESIGN

REST → estilo arquitectónico con principios específicos que, cuando se aplican con disciplina, producen APIs de alta calidad.

No es un protocolo ni un estándar técnico: es un conjunto de principios de diseño que se aplican sobre HTTP. Los principios centrales son: separación cliente-servidor, statelessness (ausencia de estado), interfaz uniforme y sistema en capas.

Stateless

El servidor no guarda ningún estado de sesión entre requests. Cada solicitud debe ser completamente autosuficiente (si requiere autenticación, debe incluir las credenciales). Beneficio: el servidor puede escalar horizontalmente y es más robusto ante fallos.

Recursos y URIs

un recurso es cualquier entidad que el sistema expone. Cada recurso tiene una URI única. Convención: las URIs son sustantivos, no verbos; la operación la determina el método HTTP. Ej: /libros es la URI de la colección, y /libros/123 es la URI de un libro específico.

CRUD y métodos HTTP

Método HTTP	Operación	URI típica	Descripción
GET	Read (lista)	/libros	Obtiene todos los libros.
GET	Read (uno)	/libros/123	Obtiene el libro con ID 123.
POST	Create	/libros	Crea un libro nuevo.
PUT	Update completo	/libros/123	Reemplaza el libro 123 completamente.
PATCH	Update parcial	/libros/123	Modifica campos específicos del libro 123.
DELETE	Delete	/libros/123	Elimina el libro con ID 123.

- HEAD: igual que GET pero sin cuerpo en la respuesta; sirve para verificar si un recurso existe o para revisar headers.
- OPTIONS: para conocer qué métodos y headers permite un endpoint; muy utilizado por los navegadores en las peticiones CORS (preflight).

Idempotencia

Un método es idempotente si ejecutarlo múltiples veces produce exactamente el mismo efecto que ejecutarlo una sola vez. GET, DELETE, PUT → idempotente. POST → no idempotente

Status codes HTTP

Son la forma en que el servidor comunica al cliente el resultado de una operación. Un cliente bien diseñado toma decisiones en función del código de estado, sin necesidad de parsear el cuerpo de la respuesta para detectar errores.

FLUJO REQUEST A RESPONSE

View → Controller → API → Middleware → Router → Handler → Store → respuesta → State → re-rende

ERRORES COMUNES

- Colocar validaciones o reglas de negocio críticas **solo en el frontend**: el backend debe validar siempre de forma independiente.
- Componentes visuales que hacen solicitudes HTTP **directamente**, sin pasar por una capa centralizada: si el endpoint cambia o se agrega autenticación, hay que rastrear y modificar

cada componente.

- Responder con **200 OK en todas las situaciones**, incluyendo errores: el cliente solo lo descubrirá si parsea el body explícitamente.
- Diseñar endpoints **con verbos** en la URI (ej: `/getLibros`) en lugar de sustantivos.
- Un handler que valida datos, aplica lógica de negocio, accede directamente a la BD y formatea la respuesta viola SRP: el handler orquesta, el Model valida, el Store accede a datos.

Termino	Definicion
MVC	Model-View-Controller. Patron arquitectonico que separa datos, presentacion y coordinacion.
Model	Componente de MVC: estructura de datos, validaciones y persistencia.
View	Componente de MVC: presentacion visual e interaccion con el usuario.
Controller / Handler	Componente de MVC: coordina View y Model; contiene la logica de aplicacion.
Router	Modulo que mapea URLs y metodos HTTP a los handlers correspondientes.
Middleware	Funcion que se ejecuta para cada request. Implementa preocupaciones transversales: logging, auth, CORS.
Store / Repository	Modulo que lee y escribe datos en el sistema de persistencia, sin logica de negocio.
SoC	Separation of Concerns. Cada modulo se ocupa de un unico tipo de preocupacion.
SRP	Single Responsibility Principle. Un modulo tiene una unica razon para cambiar.
REST	Representational State Transfer. Estilo arquitectonico con principios como statelessness e interfaz uniforme.
Stateless	Cada request es independiente y autosuficiente. El servidor no guarda estado de sesion.
Idempotencia	Una operacion que puede ejecutarse varias veces sin producir efectos distintos a la primera ejecucion.
URI	Uniform Resource Identifier. Identificador del recurso REST. Sustantivo, no verbo.
CRUD	Create, Read, Update, Delete. Las cuatro operaciones basicas sobre datos.
Status Code	Codigo HTTP en la respuesta que indica el resultado: 200, 201, 400, 404, 500, etc.
State	En el frontend, representacion en memoria de los datos actuales de la interfaz. Equivale al Model.

(4) BACKEND, PERSISTENCIA Y ORM

Lenguajes de programacion

permiten construir APIs, servicios y sistemas completos. La arquitectura (capas, persistencia, contratos HTTP, manejo de errores) es transferible.

Lenguaje	Uso común	Ventajas	Desventajas	Frameworks populares
Go	Microservicios, alto rendimiento, CLIs	Compilado, simple, concurrencia con goroutines, despliegue en binario	Ecosistema menor que Java en algunos nichos	Gin, Fiber, Echo

Java	Enterprise, sistemas grandes, integración legacy	Madurez, tooling, tipado fuerte, JVM	Más ceremonia, curva en Spring	Spring Boot, Quarkus
Python	APIs rápidas, data, scripting	Legibilidad, velocidad de desarrollo	GIL, latencia en CPU-bound puro	Django, FastAPI, Flask
Node.js	APIs, tiempo real, BFF	Mismo lenguaje que muchos frontends, async I/O	Riesgo de "callback hell" o código plano sin capas	Express, NestJS, Fastify
PHP	Web, CMS, hosting compartido clásico	Madurez en hosting, Laravel muy productivo	Historial de prácticas inconsistentes en proyectos viejos	Laravel, Symfony

2 Qué persiste y qué no

Se persiste
Estado que el dominio debe recordar

- Usuarios, pedidos, productos, pagos, configuraciones, auditoría legal.
- Refresh tokens / sesiones server-side (según diseño de autenticación).

No se persiste como negocio
Datos auxiliares de una operación

- Request HTTP cruda · flags de validación intermedios · variables locales para armar el JSON de respuesta.
- JWT stateless: la "verdad" está en la firma, no en la BD.

Guardar "por las dudas" infla la BD, aumenta superficie de datos sensibles y complica migraciones. Pregunta guía: ¿este dato se necesita en el futuro o solo para esta operación?

Otros almacenamientos:

- Caché (Redis, etc.): acelera lecturas; puede perderse; no debe ser la única fuente de verdad de datos críticos sin diseño explícito.
- Logs y métricas: persisten eventos operativos, no el modelo entidad relación del negocio.
- Blob storage: archivos (imágenes, PDF); metadatos a menudo en BD, bytes en objeto.

3 Modelo relacional aplicado al backend

- Entidad → tabla · Atributo → columna · Instancia → fila · Identidad → PK · Relación → FK o tabla puente.
- **1:N** — la FK va en el lado "muchos" (ej: `orders.user_id`).
- **N:M** — requiere tabla puente con FK hacia ambas entidades y PK compuesta.
- En código un usuario tiene `[]order` (slice); en BD no hay array: hay tabla `orders` con `user_id`. Esa diferencia es el problema objeto-relacional.
- Normalización: 1NF (atómico) · 2NF (depende de toda la clave) · 3NF (sin dependencias transitivas). Se desnormaliza por performance a veces, con costo de consistencia.

Restricciones e índices

- NOTNULL: la columna debe tener valor; útil para datos obligatorios del negocio.
- UNIQUE: no puede repetirse el valor en la tabla (ej. email de usuario).
- DEFAULT: valor si no se envía uno en el INSERT.
- CHECK: regla declarativa simple (ej. `total >= 0`); soporte variable según motor

Índices: estructura auxiliar que acelera búsquedas y algunos JOINS a costa de espacio y de lentitud en escrituras. No es gratis; mal diseñado o demasiados índices penalizan INSERT/UPDATE.

Transacciones

- **Atomicidad** — todo o nada.
- **Consistencia** — las reglas (FK, CHECK) se respetan al confirmar.
- **Aislamiento** — transacciones concurrentes no interfieren (hay niveles: READ COMMITTED, REPEATABLE READ, etc.). → más aislamiento suele costar más concurrencia.
- **Durabilidad** — una vez confirmado, el cambio persiste.

6 Migraciones de esquema

- Script versionado que transforma el esquema de estado N a N+1 de forma reproducible.
- Se versiona junto al código (mismo repo). Todos los entornos aplican la misma secuencia.
- Nunca editar migraciones ya aplicadas en producción; agregar una nueva migración correctiva.

Herramientas: Go → Goose, golang-migrate · Java → Flyway, Liquibase · Node → Knex/Prisma · Python → Alembic.

AutoMigrate del ORM solo en desarrollo/prototipo. En producción, migraciones explícitas para control y rollback.

7 El problema objeto-relacional

- Impedancia objeto-relacional: desajuste entre grafos/herencia/colecciones del paradigma OOP y tablas planas/JOINS/FK del relacional.
- Relaciones en memoria = punteros o slices. En BD = FKs y tablas puente. El backend mapea entre los dos.

- **Eager loading** — trae relaciones junto con la entidad principal (menos round-trips).
- **Lazy loading** — difiere la carga; cómodo pero provoca N+1 si no se controla.

Entender el modelo relacional es obligatorio aunque uses ORM: el ORM genera el SQL que vos diseñaste (o migraste mal).

Qué es y por qué usarlo

ORM = capa que traduce structs/objetos ↔ filas/columnas. Genera SQL parametrizado, bindea resultados, conoce metadatos (tabla, PK, FKs, tipos). No es magia: es un generador y ejecutor de SQL con reglas y convenciones.

Beneficios: menos SQL repetitivo · más productividad · menos errores de mapeo manual · portabilidad razonable entre motores.

Riesgos: SQL invisible hasta activar logs · consultas subóptimas o inesperadas · debugging indirecto ante FK violations o deadlocks · acoplamiento si el dominio queda modelado solo como anotaciones.

Qué hace el ORM por debajo

- Metadatos del modelo: tabla, columnas, PK, asociaciones, índices declarados en código.
- Genera SQL con prepared statements: los valores van como parámetros, no concatenados → mitiga inyección SQL si se usa la API correcta.
- Change tracking: sabe qué columnas cambiaron y arma UPDATE parcial. Contexto/sesión agrupa operaciones antes de enviarlas.
- Convenciones: nombre de tabla en plural snake_case, PK `id`, timestamps automáticos. Se pueden sobrescribir con tags. Ventaja: menos código repetido. Riesgo: el esquema real y el modelo en código divergen si no hay disciplina con las migraciones.

CRUD y matices

- Create → INSERT + recupera PK generada. Atención a valores por defecto en BD vs en código.
- Read → SELECT simple o con JOIN. Proyección total vs parcial de columnas.
- Update → completo o parcial. Riesgo de pisar NULLs si no se modela bien.
- Delete → físico o soft delete. Con soft delete el ORM filtra `deleted_at` automáticamente; el SQL crudo debe repetir esa condición o verá datos "fantasma".

Consultas: filtros, orden y paginación

Los ORMs ofrecen API encadenada para `WHERE`, `IN`, `LIKE`, rangos, `ORDER BY`, `LIMIT/OFFSET`, conteos y existencia. Paginación con OFFSET grande en tablas enormes puede ser costoso; en APIs públicas a veces se prefiere cursor por id/fecha.

Problema N+1 — muy pregunable

El patrón: se listan 100 usuarios con 1 query. Luego, en un bucle, se accede a `usuario.Orders` por cada uno → el ORM dispara 1 query por usuario = 101 queries en total (1 + N).

El ORM no lo arregla solo porque simplemente responde a lo que el código le pide en cada momento: cuando iterás sobre los usuarios y accedés a `usuario.Orders`, el ORM ejecuta un SELECT en ese instante, sin saber que lo vas a hacer N veces más. No tiene contexto del bucle.

Solución: Preload / eager loading (`db.Preload("Orders").Find(&users)`), JOIN explícito en una sola query, o DTO de lista que no incluya relaciones pesadas.

Eager vs lazy loading

- Eager loading: trae relaciones junto con la entidad principal. Menos round-trips si se necesita todo.
- Lazy loading: carga al acceder. Cómodo para prototipos, peligroso en listas → provoca N+1.

En APIs REST se prefiere explicitar qué incluye el endpoint (`?include=orders`) o diseñar DTOs de lista sin relaciones pesadas.

Hooks del ORM

Uso razonable: timestamps, normalización técnica, side-effects muy acotados. No meter reglas de negocio en hooks: la lógica queda invisible, es difícil de testear y se rompe con transacciones o cambios de orden de llamadas. Las reglas van en service.

Cuándo usar SQL crudo y comparativa de enfoques

SQL crudo o query builder para: reportes con `SUM / GROUP BY`, ventanas analíticas, batch masivo, optimización fina alineada a índices. Centralizar siempre en el repository.

- Driver SQL → control total, SQL a mano.
- Query builder → SQL composable sin mapeo OO completo.
- ORM → mapeo objeto-tabla + ciclo de vida + CRUD repetitivo.

Estilos de mapeo e inyección SQL

- Active Record: el modelo expone `Save()`, `Delete()`. Rápido de escribir; requiere disciplina de capas.
- Data Mapper / Repository: entidades más limpias y una capa separada que persiste. Alineado con la arquitectura por capas.

Inyección SQL: nunca concatenar input del usuario en la query. `"WHERE email = ' " + email + " '"` es peligroso. Siempre usar parámetros del prepared statement.

Idea clave

El ORM automatiza el SQL repetitivo, pero vos seguís siendo responsable del modelo de datos, de las transacciones, de la performance y de la seguridad de las consultas.

9 ORM en la práctica con Go (GORM)

- Tags en structs: `gorm:"primaryKey"` · `size:255;uniqueIndex;not null` · GORM llena `CreatedAt` / `UpdatedAt` automáticamente.
- NULL en Go: `string` no distingue NULL. Usar puntero `*string` para columnas nullable.
- Preload para evitar N+1: `db.Preload("Orders").Find(&users)` · Verificar en logs que no aparezcan cientos de SELECTs.

- Transacciones: `db.Transaction(func(tx *gorm.DB) error { ... })` · La orquestación vive en service; Transaction asegura atomicidad.
- Errores a distinguir: `record not found` (404) · violación de constraint unique/FK (409) · error de conexión (500).
- Raw SQL en repository: `db.Raw("SELECT ... WHERE created_at > ?", since).Scan(&users)` · Nunca concatenar input.

Checklist feature ORM: ¿SQL logueado es razonable? · ¿Hay N+1 en la lista? · ¿Escrituras multi-tabla en transacción? · ¿Esquema en MySQL coincide con tags? · ¿Campos sensibles no expuestos en JSON?

Arquitectura por capas

Capa	Responsabilidad	Evitar
Handler	Rutas, binding, auth HTTP superficial, respuestas	SQL directo, reglas de negocio pesadas
Service	invariantes del dominio, flujos, transacciones	detalles de columnas/tablas
Repository	persistir y recuperar por criterios técnicos	"si el usuario tiene saldo suficiente..."
Model / entidad	datos del dominio (según el estilo del proyecto)	lógica HTTP

la lógica no debe estar acoplada a strings SQL por todo el código, ni mezclada con handlers. El esquema sigue siendo crítico para performance e integridad.

Testing

- Service: test con repository mockeado (interfaz en Go) para validar reglas sin BD.
- Repository: tests de integración con BD de test o contenedor (según nivel de la materia)

11 DTO, entidades y exposición en APIs

JSON recibido → DTO de entrada → Validación → Service / entidad → Repository → DTO de salida

- No exponer siempre la entidad de BD directamente: puede tener campos internos (hashes, flags), evolucionar independiente del contrato público, o filtrar datos sensibles accidentalmente.

Estructura pensada para transportar datos en el límite del sistema (HTTP, mensajes): request body, response JSON.

12 Malas prácticas frecuentes

- 1 SQL u ORM en el handler → imposible testear negocio sin HTTP.
- 2 Reglas de negocio en el repository (edad mínima, descuentos mezclados con INSERT).
- 3 Transacciones mal delimitadas → commit parcial con estado inconsistente.
- 4 Ignorar índices y EXPLAIN hasta que producción "va lento".
- 5 N+1 al serializar listas con relaciones.
- 6 Migraciones ad hoc sin versionado compartido por el equipo.
- 7 ON DELETE CASCADE sin política de negocio clara → borrados en cadena no deseados.
- 8 Exponer el modelo de BD directamente como JSON público.
- 9 Confundir caché con fuente de verdad sin estrategia de invalidación.
- 10 Persistir todo (logs de debug, payloads enormes) en tablas de negocio.
- 11 Reglas de negocio en hooks del ORM → lógica invisible, difícil de testear.

(5) Testing de software

Permite detectar fallas antes de producción. Cuanto más tarde se detecta un bug, más caro es corregirlo: un bug encontrado con un test unitario se arregla en minutos; el mismo bug en producción puede requerir hotfix, rollback y corrección de datos.

Aporta confianza para refactorizar, documentación ejecutable (un test describe qué se espera y falla si el comportamiento cambia), mejor diseño y detección temprana. Importante: los tests no garantizan cero bugs, solo que los escenarios que decidiste probar funcionan.

Tipos:

La **pirámide de testing** describe la distribución recomendada: muchos tests unitarios (rápidos y baratos) en la base, cantidad media de integración en el medio, y pocos E2E (lentos y costosos) arriba. Tener la pirámide invertida produce suites lentas y frágiles.

- **Unit test:** verifica una función de forma aislada, sin base de datos ni HTTP. Se reemplazan dependencias externas con mocks. Ideal para lógica de negocio pura, validaciones y casos borde.
- **Integration test:** verifica que múltiples componentes funcionan juntos, incluyendo la base de datos real. Más lento pero más representativo.
- **E2E:** simula un usuario real atravesando todas las capas (frontend, backend, BD). Muy costoso y frágil; se reserva para flujos críticos.
- **API testing:** tipo de integración enfocado en los endpoints. Verifica status code, estructura del body, valores de datos, manejo de errores y headers. Es el más relevante para esta materia.

Tipo	Velocidad	Aislamiento	Confianza real	Costo
Unit	Muy rápido	Total	Bajo	Muy bajo
Integration	Medio	Parcial	Medio	Medio
API Testing	Medio	Parcial	Alto	Medio
E2E	Lento	Ninguno	Muy alto	Muy alto

Testing en APIs REST

Una API es un contrato. El testing verifica que ese contrato se cumpla.

Patrón AAA (Arrange-Act-Assert):

1. Arrange: preparar el estado inicial (datos en DB, request a construir).
2. Act: ejecutar la acción (enviar el request).
3. Assert: comparar la respuesta obtenida contra la esperada.

Qué verificar en cada respuesta: status code (lo más importante), estructura del body JSON, valores concretos de los datos, y headers relevantes.

Happy path vs unhappy path: el happy path es cuando todo va bien (inputs válidos, recursos que existen). El unhappy path es todo lo demás: inputs inválidos, recursos no encontrados, permisos insuficientes. La mayoría de los bugs reales viven en el unhappy path.

Regla práctica: por cada endpoint, cubrir al menos tres grupos: happy path, inputs inválidos, y recursos no encontrados.

Estado de la BD en tests: cada test debe preparar el estado que necesita (setup) y limpiar después (teardown) para no afectar otros tests.

Casos de prueba

Un caso de prueba formaliza qué se prueba, bajo qué condiciones y cuál es el resultado esperado. Sus elementos son: ID, descripción, precondiciones, entrada (request), resultado esperado, resultado real y estado (PASS/FAIL).

Técnicas para definir casos:

- **Partición de equivalencia:** agrupar inputs en clases de comportamiento similar. No hace falta probar todos los valores, solo uno de cada clase.
- **Análisis de valores límite:** los bugs aparecen en los extremos. Si un campo acepta 1-100, probar exactamente 1, 100, 0 y 101.
- **Happy path + unhappy path:** asegurarse de tener al menos un caso de cada tipo.

Testing manual con Postman

Postman permite hacer testing con mayor rigor que la exploración casual usando colecciones y scripts de verificación en el tab **Tests** (código JavaScript que se ejecuta después de recibir la respuesta).

js

```
pm.test("Status code es 201", function() {
  pm.response.to.have.status(201);
});
pm.test("Producto tiene id y name", function() {
  const body = pm.response.json();
  pm.expect(body).to.have.property('id');
  pm.expect(body.name).to.equal('Notebook');
});
```

Colecciones y entornos: las colecciones agrupan todos los requests. Los entornos definen variables como `{{base_url}}` para cambiar fácilmente entre desarrollo y producción.

Encadenamiento de requests: se puede guardar el ID devuelto por un POST y usarlo en el request siguiente con `pm.environment.set("product_id", body.id)`, permitiendo modelar flujos completos (POST → GET → DELETE) sin hardcodear IDs.

Limitación clave: el testing manual no es repetible de forma confiable y no se integra al proceso de desarrollo. Regla práctica: todo lo que testeas manualmente más de dos veces, automatízalo.

Testing automatizado en Go

Herramientas:

- `testing` (stdlib): soporte de testing incorporado en Go, sin instalación extra.
- `testify`: librería de assertions expresivas (`assert.Equal` , `assert.NoError` , `assert.ErrorIs`). Estándar de facto en Go.
- `net/http/httptest`: simula requests HTTP directamente contra el handler sin levantar un servidor real.

Estructura de un test en Go: los archivos terminan en `_test.go`, las funciones empiezan con `Test` y reciben `t *testing.T`. Convención de nombre: `TestFunción_Escenario_ResultadoEsperado`.

Table-driven tests: enfoque idiomático de Go para cubrir múltiples escenarios sobre la misma operación. Se define un slice de structs donde cada fila es un caso, y se ejecuta cada uno como sub-test con `t.Run`. Compacto, sin repetir el bloque AAA.

Tests de API con httptest: el flujo es siempre el mismo: crear un `ResponseRecorder`, crear el request, pasarle ambos al router con `ServeHTTP`, y verificar el código y body de la respuesta. No requiere servidor levantado.

Setup y teardown:

- `TestMain`: se ejecuta una vez antes de todos los tests del paquete. Útil para conectar a una DB de prueba.
- `t.Cleanup`: registra una función que se ejecuta cuando el test individual termina, aunque falle. Equivalente a `afterEach`.
- Regla: cada test debe dejar el sistema exactamente como lo encontró.

Comandos útiles:

```
go test ./...           → corre todos los tests
go test ./... -v         → output detallado
go test -run TestNombre → filtra por nombre
go test ./... -cover     → porcentaje de cobertura
```

Testing por capas

La arquitectura handler → service → repository no solo organiza el código: lo hace testeable de forma independiente en cada nivel.

Capa	Tipo de test	Herramienta	Requiere DB
Handler	API test	httptest	No
Service	Unit test	testing + testify + mock	No
Repository	Integration test	testing + DB de prueba	Sí

La interfaz del repository es lo que hace al service testeable. Si el repository está definido como interfaz en Go (no como struct concreta), se puede inyectar una implementación real (GORM) o una falsa (mock) según el contexto.

El mock del repository es una implementación falsa de la interfaz que guarda todo en un slice en memoria. No toca ninguna base de datos. Permite testear la lógica de negocio del service en milisegundos sin infraestructura.

Tests del service con mock: se instancia el mock con datos precargados, se crea el service pasándole el mock, y se verifica el comportamiento. Si la regla de negocio está mal, el test falla inmediatamente.

Tests del repository sí requieren una DB real. Detectan cosas que los mocks no pueden: constraints de la BD (`UNIQUE NOT NULL`), queries con errores, comportamiento real del ORM (hooks, soft delete), y que las migraciones están correctamente aplicadas. Se conecta a la DB una sola vez con `TestMain` y se limpia con `t.Cleanup` en cada test.

TDD — Test-Driven Development

TDD es una metodología donde el test se escribe **antes** que el código de producción. No es solo técnica de testing: es una forma de diseñar software. Obliga a pensar en la interfaz antes de implementarla, produciendo código más modular con dependencias más explícitas.

Ciclo Red-Green-Refactor:

-  **Red:** escribir el test que describe el comportamiento deseado. Como el código no existe, el test falla. Eso es intencional.
-  **Green:** escribir el mínimo código necesario para que el test pase. No el código perfecto, solo lo que el test exige.
-  **Refactor:** mejorar el código (renombrar, eliminar duplicación, extraer funciones) sin romper los tests. Los tests actúan como red de seguridad.

Luego se repite el ciclo para el siguiente comportamiento.

Qué cambia con TDD: el efecto más importante es el diseño. Escribir el test primero fuerza a preguntarse: ¿qué parámetros recibe la función? ¿qué devuelve cuando falla? ¿sus dependencias son inyectables? Si una función es difícil de testear, es señal de que el diseño puede mejorar.

Cuándo TDD no es ideal: exploración de un dominio desconocido, código de infraestructura y configuración, y prototipos desechables.

Buenas prácticas

- **Un test, una cosa:** cada test verifica exactamente una cosa. Si falla, debe ser obvio qué está roto.
- **Nombres descriptivos:** usar la convención `TestFunción_Escenario_ResultadoEsperado`. El nombre es documentación.
- **Tests independientes:** cada test puede correr en cualquier orden sin depender de otros. Usar setup/teardown.
- **No testear detalles de implementación:** verificar comportamiento observable (lo que devuelve la API), no cómo está implementado internamente. Un test que depende de detalles internos se rompe con cualquier refactor aunque el comportamiento no cambie.
- **Datos de prueba controlados:** definir helpers o factories de datos de prueba predecibles y representativos.
- **No ignorar tests que fallan:** un test fallido silenciado es peor que no tener el test. Si el comportamiento cambió intencionalmente, actualizar el test.
- **Saber qué no testear:** no testear getters triviales, que Go parsea JSON, o que librerías externas funcionen. Testear tu lógica: validaciones de negocio, manejo de errores propios, casos borde.
- **Cobertura:** una cobertura alta no garantiza buena calidad, pero una baja garantiza código sin testear. No buscar el 100% como fin en sí mismo; enfocarse en la lógica de negocio crítica.
- **False positive:** test que pasa aunque hay un bug real. Da falsa confianza, es el más peligroso.

- **False negative:** test que falla aunque el código es correcto. Típicamente por una assertion mal escrita.

(6) IA PARA DESARROLLADORES + SSD

¿Qué es la IA Generativa?

Large Language Models (LLMs)

- Un LLM es un sistema de IA entrenado sobre enormes cantidades de texto con un objetivo simple: **dado un texto, predecir cuál es el token siguiente más probable.**
- Un **token** ≠ una palabra. Es una unidad de ~3 a 4 caracteres.
- El modelo **no "entiende"** el código que genera: aprendió patrones estadísticos sobre billones de tokens. Cuando genera algo incorrecto con confianza, no es un bug — es el sistema funcionando como fue diseñado pero con un patrón que no se ajusta al caso.

Fase	Descripción
Entrenamiento	El modelo procesa enormes volúmenes de texto y ajusta sus parámetros para minimizar el error de predicción. Ocurre una sola vez.
Inferencia	Cuando el usuario hace una pregunta, el modelo genera tokens uno a uno eligiendo el más probable. Ocurre cada vez que se manda un mensaje.

Consecuencia clave: el modelo no busca en internet en tiempo real, no accede al repositorio y no recuerda conversaciones anteriores. **Todo lo que sabe del problema está en el contexto que se le da.**

La Ventana de Contexto

- Es el conjunto de texto que el modelo puede considerar al generar una respuesta: prompt, historial, archivos pegados y la respuesta que va generando.
- Tiene un límite. **Todo lo que no está en el contexto, la IA no lo sabe.**

La IA SÍ sabe	La IA NO sabe (salvo que se lo pongas)
Lo que pusiste en el prompt	El resto del repositorio
El historial de la conversación actual	Decisiones de arquitectura anteriores
Patrones de código del entrenamiento	Las convenciones específicas del equipo
APIs y librerías conocidas (hasta su fecha de corte)	Cambios recientes en dependencias
Lo que le pegás en el mensaje	El contexto de negocio de la aplicación

¿Por qué ocurren las alucinaciones?

- El modelo está optimizado para generar **texto probable**, no texto verdadero.
- **La detección de alucinaciones es responsabilidad del desarrollador.** Verificar funciones, métodos y APIs en la documentación oficial es parte del flujo de revisión, no una actividad opcional.

Herramientas de IA para Desarrollo

Categoría	Descripción	Ejemplos	Cuándo usar
Asistentes de chat	Conversación libre, pedidos explícitos, contexto manual. El	Claude, ChatGPT, Gemini	Tareas con mucho contexto, explicaciones, generaciones complejas

Categoría	Descripción	Ejemplos	Cuándo usar
	dev formula el pedido, lee el output completo y decide qué incorporar.		
Autocompletado en editor	Sugieren mientras escribís, contexto = archivo actual	GitHub Copilot, Cursor	Boilerplate rápido, patrones repetitivos
Agentes de código	Leen el repo completo, ejecutan comandos, hacen cambios autónomos	Claude Code, Devin, Aider	Refactors grandes, tareas multi-archivo (más supervisión)
Integrados en CI/CD	IA en code review, generación de tests, documentación automática	CodeRabbit, Mintlify	Complemento al review, no reemplazo

Lo que cambia y lo que NO cambia con la IA

Lo que CAMBIA	Lo que NO CAMBIA
La velocidad para generar un primer borrador	La responsabilidad del dev sobre el código que commitea
El costo de explorar alternativas de implementación	La necesidad de entender lo que se escribe
La facilidad para entender código ajeno	La importancia de especificar bien el problema
El tiempo invertido en boilerplate repetitivo	El criterio para elegir la arquitectura correcta
La accesibilidad a patrones y convenciones comunes	La habilidad para detectar bugs y vulnerabilidades

Spec-Driven Development (SDD)

¿Qué es?

Práctica de escribir una **especificación clara y detallada del sistema antes de escribir una sola línea de código**, y de mantenerla como fuente de verdad durante todo el ciclo de desarrollo. No es una metodología nueva. Lo que cambió es que hoy la especificación cumple un **doble rol**:

1. Es la guía para el equipo humano.
2. Es el contexto que se le da a la IA para que genere código relevante.

¿Qué es una especificación?

Documento que describe de forma **precisa y sin ambigüedades qué debe hacer el sistema**, sin prescribir cómo implementarlo. Define el contrato entre el sistema y sus usuarios.

Una buena especificación responde:

- ¿Qué hace este endpoint / función / módulo? (*descripción en lenguaje de negocio*)
- ¿Cuáles son las entradas? (*tipos, validaciones, restricciones, opcionales vs obligatorios*)
- ¿Cuáles son las salidas? (*estructura de respuesta, tipos, campos posibles*)
- ¿Cuáles son los casos de error? (*código de error, mensaje*)

- ¿Cuáles son las reglas de negocio? (*invariantes que siempre deben cumplirse*)

Formatos de especificación

Formato	Cuándo usarlo	Herramienta
OpenAPI / Swagger	APIs REST: endpoints, schemas, respuestas	Swagger Editor, Stoplight
AsyncAPI	APIs orientadas a eventos, mensajería, WebSockets	AsyncAPI Studio
User Stories + Acceptance Criteria	Funcionalidades desde la perspectiva del usuario	Jira, Linear, Notion
PRD	Descripción completa de un producto o feature	Confluence, Notion, Google Docs
ADR	Decisiones de arquitectura y su contexto	Repositorio Git

La spec como contrato entre equipos

- Permite que **frontend y backend trabajen en paralelo**: el frontend sabe exactamente qué va a recibir antes de que el backend exista.
- El equipo de QA puede escribir los tests antes de que exista el código.
- Sin spec, cada cambio en la API se convierte en una conversación informal con riesgo de malentendidos.

El Flujo de Trabajo con IA

Paso	Quién	Descripción
1. Especificación	Desarrollador	Escribir la spec. Define QUÉ hace el sistema.
2. Prompt con contexto	Dev + IA	Se le da la spec a la IA con un pedido específico y acotado.
3. Generación	IA	La IA produce código, tests, documentación. Es un borrador .
4. Revisión	Desarrollador	Verificar seguridad, correctitud, coherencia con la spec y el sistema. no puede delegarse → + crítico
5. Refactor	Desarrollador	Ajustar, limpiar, adaptar al estilo del proyecto. El código que entra al repo es del dev.
6. Integración	Desarrollador	Integrar con el resto del sistema, correr tests, hacer code review.

Granularidad del prompt

-  **Muy grande**: "Generá el backend completo de la librería" → produce algo genérico.
-  **Correcto**: "Generá el handler de `POST /libros` en Go con Gin, usando estos tipos y esta lógica de validación".
-  **También correcto**: "Dado este handler, generá los table-driven tests para los casos de error".

La IA como Par de Programación

Buena para:

- Boilerplate y código repetitivo (CRUD, routers, serialización).

- Transformaciones de formato (spec OpenAPI → structs Go, JSON → tipos tipados, SQL a partir de modelo).
- Generación de tests básicos (happy path y casos de validación obvios).
- Explicación de código y patterns desconocidos.
- Refactor de funciones acotadas (renombrar, extraer funciones, simplificar condicionales).
- Documentación (godoc, comentarios, README básico).

Mala para:

- Entender el negocio y las reglas de dominio.
- Garantizar seguridad (puede generar código vulnerable sin aviso).
- Recordar el contexto completo en sistemas grandes.
- Detectar sus propias alucinaciones.
- Decidir arquitecturas (requiere entender trade-offs reales).
- Validar que su propio output es correcto.

Prompt Engineering para Desarrollo

1. Dar contexto — sin contexto, produce algo genérico.

2. Ser específico sobre el output — indicar exactamente qué forma debe tener el resultado.

3. Iterar en lugar de pedir todo de una vez — el primer resultado es un borrador. Pedir algo pequeño, revisar y pedir el siguiente paso es más efectivo que pedir todo el sistema de una sola vez.

Riesgos de la IA en el Desarrollo

- Alucinaciones
- Código inseguro → Reproduce los patrones que vio, incluyendo los malos.
- Dependencias innecesarias
- Over-engineering

La IA cambia el rol del desarrollador: pasa de escribir todo el código a **diseñar el sistema, especificar con precisión, dirigir la generación y ser el árbitro final de la calidad.**

Responsabilidades que no se delegan:

1. Definir qué construir: especificación, diseño de la arquitectura y decisiones de negocio.

2. Evaluar el output: Todo código que entra al repositorio es responsabilidad del desarrollador que lo aprueba.

3. Mantener el sistema: Debe ser legible, documentado y coherente con el resto del sistema.

(7) DESIGN PATTERNS, DAO Y ANTIPATRONES

Un patron de diseno es una solucion general y reutilizable a un problema recurrente en el diseno orientado a objetos. Los patrones no se aplican, se reconocen. Cuando el codigo empieza a tener un problema conocido, el patron provee el nombre y la solucion

- "Strategy" comunica una decision de diseno completa sin mostrar el codigo.

- Trade-offs explicitos: cada patron tiene un costo (mas indireccion, mas abstraccion). Conocer el patron implica saber cuando no usarlo.
- Experiencia codificada: son soluciones probadas en miles de proyectos reales.

Categoria	Proposito	Patrones principales
Creacionales	Como se crean los objetos. Desacoplan la construccion del uso.	Factory Method, Abstract Factory, Singleton, Builder, Prototype
Estructurales	Como se componen los objetos para formar estructuras mas grandes.	Adapter, Decorator, Facade, Composite, Proxy, Bridge, Flyweight
Comportamiento	Como se comunican y coordinan los objetos entre si.	Strategy, Observer, Template Method, Command, Iterator, Chain of Resp, State

Patrones creacionales

- Factory method: define una interfaz para crear un objeto, pero deja que el llamador decida que clase instanciar. El codigo que usa el objeto no necesita conocer el tipo concreto. Agregar un nuevo canal (SMS, Teams) significa crear una nueva struct, sin modificar el codigo que usa los notificaciones.

```
// Interfaz comun
type Notifier interface { Send(msg string) error }

// Factory: el caller recibe la interfaz, no el tipo concreto
func NewNotifier(cfg Config) Notifier {
    switch cfg.Channel {
    case "email": return &EmailNotifier{addr: cfg.Target}
    case "slack": return &SlackNotifier{webhook: cfg.Target}
    default:     return &NoopNotifier{}
    }
}
```

- Singleton: Garantiza que una clase tenga una unica instancia y provee un punto de acceso global. En Go se implementa con sync.Once para thread-safety. El estado global dificulta el testing. La alternativa preferida en Go es crear la instancia una vez en main() e inyectarla explicitamente como dependencia.

```
var (once sync.Once; instance *DB)

func GetDB() *DB {
    once.Do(func() { instance = &DB{ /* inicializar */ } })
    return instance
}
```

Patrones estructurales

- Adapter: Convierte la interfaz de una clase en otra que el cliente espera. Permite que componentes con interfaces incompatibles trabajen juntos sin modificar ninguno de los dos.

```
// Tu interfaz interna
type Mailer interface { Send(to, subject, body string) error }

// Adapter: implementa Mailer, usa SendGrid internamente
type SendGridAdapter struct { client *sendgrid.Client }
func (a *SendGridAdapter) Send(to, subject, body string) error {
    _, err := a.client.SendEmail(sendgrid.Request{To: to, Subject: subject, Body: body})
    return err
}
```

- Decorator: Agrega comportamiento a un objeto envolviendolo en otro que implementa la misma interfaz. El cliente no sabe si esta hablando con el objeto original o con el decorator.

```
// Decorator: agrega retry a cualquier Mailer
type RetryMailer struct { inner Mailer; retries int }

func (r *RetryMailer) Send(to, subject, body string) error {
    var err error
    for i := 0; i < r.retries; i++ {
        if err = r.inner.Send(to, subject, body); err == nil { return nil }
    }
    return err
}
// Se pueden apilar: Retry(Log(SendGridAdapter))
```

- Facade: Provee una interfaz simplificada a un subsistema complejo. El cliente solo conoce la facade; los internos quedan ocultos y pueden cambiar sin afectar al cliente.

```
type OrderFacade struct {
    stock    *StockService
    payment  *PaymentService
    shipping *ShippingService
    notifier Notifier
}
```

```
func (f *OrderFacade) PlaceOrder(order Order) error {
    if err := f.stock.Reserve(order); err != nil { return err }
    if err := f.payment.Charge(order); err != nil { return err }
    f.shipping.Schedule(order)
    f.notifier.Send("Orden confirmada")
    return nil
}
```

Patrones de comportamiento

- Strategy: Define una familia de algoritmos, encapsula cada uno y los hace intercambiables. El objeto que los usa no sabe cual esta empleando en runtime. Agregar una nueva estrategia de precios no requiere tocar Cart ni ninguna estrategia existente.

```
type PricingStrategy interface { Calculate(base float64) float64 }

type RegularPrice struct{}
type StudentDiscount struct{}
type BulkDiscount struct{ Min int }

func (r RegularPrice) Calculate(b float64) float64 { return b }
func (s StudentDiscount) Calculate(b float64) float64 { return b * 0.8 }
func (k BulkDiscount) Calculate(b float64) float64 { return b * 0.7 }

type Cart struct { pricing PricingStrategy }
func (c *Cart) Total(base float64) float64 { return c.pricing.Calculate(base) }
```

- Observer: Define una relacion uno-a-muchos: cuando un objeto cambia su estado, todos sus dependientes son notificados automaticamente. Desacopla al emisor de los receptores

```
type Observer interface { OnEvent(event Event) }

type EventBus struct { handlers []Observer }

func (b *EventBus) Subscribe(o Observer) {
    b.handlers = append(b.handlers, o)
}

func (b *EventBus) Publish(e Event) {
    for _, h := range b.handlers { h.OnEvent(e) }
}
```

- Template method: Define el esqueleto de un algoritmo, dejando que los pasos variables sean provistos por quien usa la estructura. El orden del algoritmo no cambia.

```
type ReportGenerator struct {
```

```

FetchData func() []Row
FormatData func([]Row) string
OutputDest func(string) error
}

// El template: el orden nunca cambia
func (g ReportGenerator) Run() error {
    rows := g.FetchData()
    formatted := g.FormatData(rows)
    return g.OutputDest(formatted)
}

```

Patron DAO/ REPOSITORY

El patron Data Access Object (DAO), tambien llamado Repository, encapsula toda la logica de acceso a la base de datos en un objeto dedicado. El resto del sistema solo habla con la interfaz, sin conocer como se persisten los datos.

sin DAO:

```

// Sin DAO: SQL en el handler
func GetUser(w http.ResponseWriter, r *http.Request) {
    id := getParam(r, "id")
    var u User
    db.QueryRow("SELECT * FROM users WHERE id=?", id).Scan(&u.ID, &u.Name,
    &u.Email)
    json.NewEncoder(w).Encode(u)
}

```

con DAO:

```

// Interfaz pura: sin SQL ni ORM
type UserRepository interface {
    FindByID(id int) (User, error)
    Save(u User) error
    Delete(id int) error
}

// Handler limpio: solo HTTP
func (h *Handler) GetUser(w http.ResponseWriter, r *http.Request) {
    id := getParam(r, "id")
    u, err := h.users.FindByID(id)
    if err != nil { http.Error(w, "not found", 404); return }
    json.NewEncoder(w).Encode(u)
}

```

el handler es testeable con un mock. La implementacion concreta (GORM, SQL raw, in-memory) puede cambiar sin tocar el handler.

ANTIPATRONES

Respuesta comun a un problema recurrente que parece razonable pero produce consecuencias negativas a largo plazo. A diferencia de un error obvio, los antipatrones se instalan lentamente y dannan con el tiempo.

- God object: Una clase o struct que sabe demasiado y hace demasiado. Acumula responsabilidades que deberian estar separadas.

```
// God Object: una struct con 8 dependencias y 40 metodos
type UserManager struct {
    db, cache, mailer, metrics, logger, validator, tokenSvc, cryptoSvc interface{}
}
// CreateUser, SendEmail, CacheUser, HashPassword, GenerateToken...

// Correcto: una responsabilidad por struct
type UserService struct { repo UserRepository; mailer Mailer; cache Cache }
```

- Señal de alarma: struct con mas de ~5 dependencias o mas de ~10 metodos
- Impacto: un cambio en cualquier dependencia puede romper funcionalidades no relacionadas
- Solucion: Single Responsibility Principle (SRP)
- Spaghetti code:

Flujo de control tan enredado que es imposible seguirlo: condicionales anidadas muy profundas, flags booleanos en cadena, funciones que hacen demasiadas cosas.

- Surge cuando se agrega logica ad-hoc sin refactorizar
- Impacto: imposible agregar funcionalidades o corregir bugs sin romper otra cosa
- Solucion: early returns, separacion de concerns, funciones pequenas con un unico proposito
- Logica de negocio en el controlador

El handler HTTP contiene validaciones, reglas de negocio, queries a la DB y formateo de la respuesta. El resultado es codigo que no se puede reusar ni testear de forma aislada.

- Acceso directo a la base de datos

SQL hardcodeado en handlers o services sin ninguna capa de abstraccion. Imposible de testear en aislamiento y muy dificil de migrar a otro motor de base de datos

- Impacto: cada cambio en el esquema requiere buscar y modificar SQL en todo el codigo
- Impacto: los tests requieren una DB real en ejecucion
- Solucion: patron DAO / Repository con interfaz que oculta el mecanismo de persistencia

Señal en el codigo	Patron a considerar	Beneficio
switch que crece con cada nuevo tipo	Factory Method / Strategy	Nuevo tipo = nueva struct, sin tocar el resto
SQL en el handler o service	DAO / Repository	Handler testeable, DB intercambiable
Libreria externa con interfaz distinta	Adapter	Tu codigo no depende de la interfaz externa
Quiero agregar retry/log sin modificar clase	Decorator	Comportamiento extra sin modificar el original
Handler orquesta subsistema complejo	Facade	Interfaz simple, internos ocultos
Struct con 7+ dependencias	Revisar SRP	Dividir = mas mantenible y testeable

(8) SEGURIDAD, OWASP, AUTENTICACION

¿Por qué importa la seguridad?

Los ataques a aplicaciones web son cotidianos y automatizados. No importa el tamaño del sistema: cualquier API expuesta a internet puede ser atacada en segundos. La diferencia entre un sistema seguro y uno vulnerable está en si el desarrollador conocía y aplicó los conceptos básicos.

Costo de no diseñar con seguridad: corregir una vulnerabilidad en producción es entre 6 y 30 veces más costoso que haberla prevenido. Hay que parchear, investigar si fue explotada, notificar a usuarios (obligatorio por ley en muchos países) y restaurar la confianza.

Seguridad desde el diseño

La seguridad empieza antes de codear, cuando se decide qué datos existen, quién puede modificarlos y qué supuestos se hacen sobre el cliente.

Cuatro preguntas antes de implementar cualquier funcionalidad:

Pregunta	Ejemplo
¿Qué activo protegemos?	pedidos, datos personales, tokens, pagos
¿Quiénes interactúan con el sistema?	usuario, admin, servicio interno, atacante anónimo
¿Qué datos controla el cliente?	IDs en la URL, body JSON, headers, cookies
¿Qué debe verificar el servidor?	identidad, permisos, ownership, límites de uso

Límites de confianza: todo dato que viene del cliente cruza un límite de confianza. La regla es simple: todo dato externo se valida, se autoriza y se trata como no confiable. Esto incluye IDs, roles, emails, precios, archivos, headers y cookies.

Defensa en profundidad: no existe un único control que haga segura una aplicación. Se combinan capas: autenticación, autorización, validación de input, queries parametrizadas, hashing de passwords, HTTPS, secrets fuera del código, logs sin datos sensibles, rate limiting y tests. Si una capa falla, las demás reducen el impacto.

Autenticación vs Autorización

Son conceptos completamente distintos y confundirlos lleva a errores graves de diseño.

Concepto	Pregunta que responde	Ejemplo
Autenticación	¿Quién sos?	"Soy <u>juan@mail.com</u> y esta es mi contraseña"
Autorización	¿Qué podés hacer?	"Juan puede ver sus pedidos pero no los de otros"

- **Autenticación** ocurre una vez, en el login. Verifica identidad.
- **Autorización** ocurre en cada request, después de autenticar. Verifica permisos.

Códigos HTTP:

- **401 Unauthorized** → no autenticado (falta token, token expirado, credenciales incorrectas)
- **403 Forbidden** → autenticado pero sin permiso para esa acción
- `GET /orders/99` sin token → **401**
- `GET /orders/99` con token de otro usuario → **403** o **404** (404 es mejor práctica porque no revela si el recurso existe)
- `GET /admin/reports` sin token → **401**
- `GET /admin/reports` con token de usuario común → **403**
- `DELETE /products/5` con token de usuario sin rol "seller" → **403**

OWASP Top 10

OWASP es una organización sin fines de lucro dedicada a mejorar la seguridad del software que publica el ranking de los riesgos de seguridad más críticos. Es el estándar de referencia de la industria.

OWASP Top 10 — 2021:

#	Riesgo	Descripción
A01	Broken Access Control	Usuarios que actúan fuera de sus permisos
A02	Cryptographic Failures	Datos sensibles expuestos por cifrado débil o ausente
A03	Injection	Input del usuario interpretado como comandos (SQL, etc.)
A04	Insecure Design	Fallas de diseño que no se pueden corregir solo con implementación
A05	Security Misconfiguration	Configuraciones por defecto inseguras, errores expuestos
A06	Vulnerable and Outdated Components	Dependencias con vulnerabilidades conocidas sin parchear
A07	Identification and Authentication Failures	Passwords débiles, sesiones sin expirar
A08	Software and Data Integrity Failures	CI/CD inseguros, deserialización sin validación
A09	Security Logging and Monitoring Failures	Sin logs ni alertas, ataques que pasan desapercibidos
A10	Server-Side Request Forgery (SSRF)	El servidor hace requests a destinos controlados por el atacante

OWASP API Security Top 10 (versión enfocada en APIs REST):

Riesgo	Idea central
Broken Object Level Authorization	Un usuario accede a objetos de otro cambiando un ID
Broken Authentication	Tokens o credenciales mal protegidas
Broken Object Property Level Authorization	El cliente lee o modifica campos que no debería
Unrestricted Resource Consumption	Un endpoint permite consumir demasiados recursos
Broken Function Level Authorization	Funciones administrativas accesibles por usuarios comunes
Unsafe Consumption of APIs	Se confía demasiado en datos recibidos desde otros servicios

En APIs, la pregunta clave no es solo "¿está autenticado?" sino "**¿este usuario puede hacer esta acción sobre este recurso específico?**"

Broken Access Control (A01)

Es el riesgo número 1 de OWASP. Ocurre cuando los controles de autorización son incorrectos, incompletos o inexistentes.

```
// VULNERABLE – ¿por qué?  
// Solo usa el ID de la URL, no verifica que el pedido sea del usuario  
db.First(&order, id)
```

```
// SEGURO – ¿por qué?  
// Filtra por ID Y por userID del token  
db.Where("id = ? AND user_id = ?", id, userID).First(&order)
```


Devolver **404** en lugar de 403 cuando el recurso existe pero pertenece a otro usuario es la mejor práctica porque no revela información sobre el sistema.

Tipos comunes:

IDOR (Insecure Direct Object Reference): el usuario accede a recursos de otros cambiando un ID en la URL.

```
GET /api/orders/1234 → ve su pedido ✓  
GET /api/orders/1235 → ve el pedido de otra persona ✗
```

El backend usó el ID sin verificar si el usuario autenticado es el dueño.

Privilege Escalation: un usuario básico accede a funcionalidad de administrador.

Broken Object Property Level Authorization: el usuario puede modificar un recurso pero no todos sus campos. Si el backend guarda todo lo que el cliente manda sin filtrar, el cliente puede modificar campos que solo debería controlar el servidor (ej: cambiar su propio `role` a `"admin"`).

Forzado de navegación: acceder directamente a una URL protegida que el frontend simplemente no muestra, pero el backend tampoco verifica.

La autorización combina tres preguntas:

Pregunta	Ejemplo
Rol	¿Es admin, seller o user?
Ownership	¿El recurso pertenece a este usuario?
Estado	¿El pedido está en un estado que permite esta acción?

Regla de oro: todo control de acceso se verifica en el backend, siempre. El frontend nunca es la fuente de verdad sobre permisos. El middleware por rol es útil pero no reemplaza la verificación de ownership dentro del handler.

SQL Injection (A03)

Ocurre cuando input del usuario se inserta directamente en una query SQL sin ser tratado como dato. El atacante puede modificar la lógica de la consulta.

Ejemplo clásico: si el sistema arma la query concatenando strings y el atacante ingresa `' OR '1'='1` como email, la query resultante siempre es verdadera y devuelve el primer usuario de la tabla sin necesidad de credenciales válidas. Con `'; DROP TABLE users; --` se puede destruir tablas enteras.

La defensa es absoluta: nunca concatenar input del usuario en SQL. Siempre usar queries parametrizadas (prepared statements). El driver de base de datos separa el código SQL del dato, por lo que el input malicioso nunca se interpreta como instrucción.

Importante con ORMs: GORM protege contra SQL Injection en sus métodos estándar (`Where`, `Find`, `Create`), pero el riesgo vuelve si se usa `db.Raw()` con concatenación de strings. `db.Raw()` es seguro solo si se usan `?` como parámetros.

El mismo principio aplica a otros tipos de injection: NoSQL Injection (MongoDB), OS Command Injection, LDAP Injection, Template Injection. La defensa siempre es separar datos de instrucciones.

```
// VULNERABLE — concatenación directa  
query := "SELECT * FROM users WHERE email='" + email + "'" +  
db.Raw(query).Scan(&user)  
  
// SEGURO — parámetros con ?  
db.Raw("SELECT * FROM users WHERE email = ?", email).Scan(&user)
```

// SEGURO — método estándar de GORM

```
db.Where("email = ?", email).First(&user)
```

// VULNERABLE — aunque use GORM, la concatenación lo rompe

```
query := fmt.Sprintf("SELECT * FROM users WHERE name = '%s'", nombre)
```

```
db.Raw(query).Scan(&result)
```

XSS y CSRF

XSS — Cross-Site Scripting

Ocurre cuando un atacante logra que el navegador de otra persona ejecute JavaScript malicioso. Ataca al cliente, no al servidor.

- "Un comentario con `<script>` que roba cookies de todos los que ven la página" → **Stored XSS**
- "Un link que al hacer click ejecuta una transferencia en el banco donde estás logueado" → **CSRF**
- "Un parámetro de búsqueda que refleja código JavaScript en la respuesta" → **Reflected XSS**
- "La cookie no tiene `HttpOnly` y un script malicioso lee `document.cookie`" → **consecuencia de XSS**

Tipos:

- **Reflected XSS:** el script viene en la request y se refleja en la respuesta (ej: un link trampa con código en un query param).
- **Stored XSS:** el script se guarda en la base de datos (ej: en un comentario) y se ejecuta para todos los que visiten esa página. Es el más peligroso.
- **DOM-based XSS:** la manipulación ocurre en el JavaScript del cliente sin pasar por el servidor.

Qué puede hacer un atacante con XSS: robar cookies de sesión, redirigir a phishing, hacer requests en nombre del usuario, capturar lo que escribe (keylogging), modificar el contenido de la página.

Cómo prevenir XSS:

1. **Escapar todo output:** React hace esto automáticamente en JSX. `dangerouslySetInnerHTML` desactiva el escape y debe usarse solo con contenido propio sanitizado.
2. **Content Security Policy (CSP):** header HTTP que le dice al navegador de qué orígenes puede cargar scripts.
3. **Flags en cookies:** `HttpOnly` hace que la cookie no sea accesible desde JavaScript. `Secure` hace que solo se envíe por HTTPS.

CSRF — Cross-Site Request Forgery

Engaña al navegador del usuario para que haga una request autenticada a otra aplicación sin que el usuario lo sepa. Explota que el navegador adjunta cookies automáticamente en cualquier request al dominio correspondiente.

Ejemplo: una imagen invisible en evil.com cuya URL es `https://banco.com/transferir?`

`monto=5000&destino=atacante`. Si el usuario está logueado en banco.com, el navegador manda sus cookies y el banco procesa la transferencia.

Diferencia con XSS:

- XSS → el atacante ejecuta código en el navegador de la víctima.

- CSRF → el atacante hace que el navegador ejecute requests autenticadas.

Cómo prevenir CSRF:

1. **SameSite cookies:** `Strict` no envía cookies en ninguna request cross-site. `Lax` las envía solo en navegación top-level. Los navegadores modernos usan `Lax` por defecto, lo que ya mitiga muchos ataques.
2. **CSRF Token:** el servidor genera un token aleatorio que se incluye en formularios y se verifica en cada POST. Un atacante en otro dominio no puede leer ese token.
3. **Verificar el header `Origin`:** el servidor comprueba que el origen de la request sea uno permitido.

Nota importante: las APIs que usan JWT en el header `Authorization` (no en cookies) reducen el riesgo de CSRF, porque el navegador no adjunta ese header automáticamente en requests cross-site. Pero si el JWT está en una cookie, vuelven a importar todas las defensas contra CSRF.

Hashing de passwords: bcrypt

¿Por qué no guardar passwords en texto plano? Si la base de datos se filtra, los passwords de todos quedan expuestos. Y como la mayoría reutiliza passwords, el atacante accede también a otros sistemas.

¿Por qué no alcanza con cifrado? El cifrado es reversible si se tiene la clave. La solución es el hash unidireccional: una función matemática que no puede revertirse.

¿Por qué no alcanza con SHA-256 o MD5? Son extremadamente rápidos: con una GPU moderna se pueden probar miles de millones de contraseñas por segundo. bcrypt fue diseñado específicamente para passwords y tiene dos propiedades fundamentales:

1. **Es lento por diseño:** tiene un parámetro `cost` (o work factor) que controla las iteraciones. Con `cost=12`, hashear tarda ~250ms, imperceptible para el usuario pero que hace inviable el ataque de fuerza bruta.
2. **Incluye un salt automático:** valor aleatorio que se agrega al password antes de hashear. Dos usuarios con el mismo password tendrán hashes distintos. Esto invalida los ataques con rainbow tables (tablas precomputadas de hashes).

Flujo correcto:

- Al registrar: hashear el password con bcrypt y guardar el hash. Nunca guardar el password original.
- Al hacer login: usar `bcrypt.CompareHashAndPassword` para comparar el password ingresado con el hash guardado.

El hash de bcrypt incluye embebido el salt y el cost factor, por lo que no hace falta guardarlos por separado.

En el login, siempre devolver el mismo mensaje de error tanto si el usuario no existe como si el password es incorrecto. No revelar cuál fue el fallo exacto evita que un atacante enumere usuarios válidos.

Sesiones vs Tokens

Sesiones (stateful)

El servidor guarda el estado de la sesión en memoria o base de datos. El usuario recibe un ID de sesión (en cookie) y en cada request el servidor busca ese ID.

Ventajas: fácil de invalidar (borrás la sesión y el usuario queda deslogueado al instante). Control total del servidor sobre sesiones activas.

Desventajas: el servidor debe guardar estado (viola REST stateless). No escala bien horizontalmente: si hay varios servidores, todos necesitan acceder al mismo session store compartido (Redis, etc.). Cada request implica una consulta al store.

Tokens (stateless)

El servidor genera un token firmado con toda la información del usuario. El usuario lo guarda y lo envía en cada request. El servidor solo verifica la firma, sin consultar ninguna base de datos.

Ventajas: stateless, escala horizontalmente sin session store. Portable entre dominios y servicios (ideal para microservicios y apps móviles).

Desventajas: no se puede invalidar fácilmente antes de que expire. Si la clave de firma se compromete, todos los tokens son inválidos.

Criterio	Sesiones	Tokens (JWT)
App web tradicional monolítica	✓	✓
API REST con múltiples clientes	✓	✓
Microservicios	✗	✓
App móvil	✗	✓
Invaldar sesiones al instante	✓	Complejo
Escala horizontal sin coordinación	✗	✓

- ✗ "bcrypt es reversible con la clave" → falso, es unidireccional
- ✗ "MD5 es suficiente para passwords porque también es un hash" → falso, es demasiado rápido
- ✗ "el salt hay que guardarlo por separado" → falso, está embebido en el hash
- ✓ "dos usuarios con el mismo password tienen hashes distintos gracias al salt" → verdadero
- ✓ "aumentar el cost factor hace el hash más lento y más seguro contra fuerza bruta" → verdadero

JWT — JSON Web Token

JWT (RFC 7519) es el estándar para tokens stateless. Es un formato compacto para transmitir información firmada digitalmente.

Estructura

Un JWT tiene tres partes separadas por puntos:

1. Header — tipo de token y algoritmo de firma:

json

```
{ "alg": "HS256", "typ": "JWT" }
```

2. Payload (Claims) — datos del token:

json

```
{  
  "sub": "42",           // subject: ID del usuario  
  "iss": "api.tienda.com", // issuer: quién lo emitió  
  "aud": "app.tienda.com", // audience: para quién es  
  "exp": 1716828800,      // expiration: Unix timestamp de expiración  
}
```

```

"iat": 1716825200, // issued at: cuándo se emitió
"role": "admin" // claim personalizado
}

```

3. Signature — garantiza que el token no fue modificado. Se calcula aplicando HMAC-SHA256 al header y payload codificados, usando una clave secreta. Si alguien modifica el payload, la firma no coincide y el token es rechazado.

Importante: el payload está codificado en Base64URL, NO cifrado. Cualquiera puede decodificarlo y leerlo. **Nunca incluir datos sensibles en el payload** (passwords, números de tarjeta, etc.).

Dónde guardar el token en el cliente

Lugar	Ventaja	Riesgo
Memoria	No persiste al cerrar la pestaña	Se pierde al refrescar
localStorage	Simple y persistente	Un XSS puede leerlo
Cookie HttpOnly	JavaScript no puede leerla	Requiere defensas CSRF

No hay respuesta universal. Para proyectos académicos: entender el trade-off. localStorage simplifica pero amplifica el impacto de XSS. Cookie HttpOnly protege contra robo por JS pero exige pensar en CSRF.

Flujo completo con JWT

1. **Registro:** el cliente manda email y password. El servidor hashea el password con bcrypt y guarda el usuario.
2. **Login:** el cliente manda credenciales. El servidor busca el usuario, compara el password con bcrypt, y si coincide genera un JWT firmado y lo devuelve.
3. **Requests autenticadas:** el cliente manda el JWT en el header `Authorization: Bearer <token>`. El servidor verifica la firma y extrae el userID sin consultar ninguna base de datos. Luego sí consulta la base de datos con ese userID para obtener los datos del recurso.

Refresh Tokens

El problema de JWT es que no se puede invalidar antes de que expire. La solución es usar dos tokens:

- **Access Token:** JWT de corta duración (15 min a 1 hora). Se usa en cada request. Si se compromete, el daño es limitado por el tiempo de expiración.
- **Refresh Token:** de larga duración (días o semanas), guardado en base de datos. Se usa solo para obtener un nuevo access token cuando el actual expira.

Para "cerrar sesión": se borra el refresh token de la base de datos. El access token sigue válido hasta que expira, pero no puede renovarse.

Middleware JWT en Go (lógica)

El middleware intercepta cada request protegida, extrae el token del header `Authorization: Bearer <token>`, valida la firma y la expiración, y si todo es correcto guarda el userID y el rol en el contexto de la request para que los handlers los usen sin consultar la base de datos.

OAuth 2.0 y OpenID Connect

OAuth 2.0 es un protocolo de autorización que permite a una aplicación obtener acceso limitado a recursos de un usuario en otro servicio, sin que el usuario comparta sus credenciales. El ejemplo más familiar: "Iniciar sesión con Google".

Importante: OAuth 2.0 es de autorización, no de autenticación. Cuando se usa para login, se combina con **OpenID Connect (OIDC)** que agrega una capa de identidad.

Tecnología	Responde
OAuth 2.0	¿Esta aplicación puede acceder a este recurso?
OpenID Connect	¿Quién es este usuario?

Roles en OAuth 2.0:

Rol	Descripción	Ejemplo
Resource Owner	El usuario que posee los datos	Juan
Client	La app que quiere acceder	Tu app "MiTienda"
Authorization Server	Quien autentica y emite tokens	Google, GitHub
Resource Server	Donde viven los datos	Google APIs

Flujo Authorization Code (el más usado en apps con backend):

1. El usuario hace click en "Login con Google".
2. La app redirige al usuario a Google con `client_id`, `scope` y `state`.
3. El usuario se autentica en Google y autoriza los permisos solicitados.
4. Google redirige de vuelta a la app con un `code` temporal.
5. El backend intercambia ese `code` por tokens usando el `client_secret`.
6. El backend obtiene los datos del usuario (email, nombre) y crea o busca el usuario en su propia base de datos.
7. El backend genera su propio JWT y se lo devuelve al cliente.

Por qué generar nuestro propio JWT: el token de Google es válido para las APIs de Google, no para la nuestra. Para la sesión de nuestra app necesitamos nuestro propio token con nuestros propios datos (ID local, roles, permisos).

Puntos de seguridad en OAuth:

- El parámetro `state` debe ser aleatorio y verificado en el callback (protege contra CSRF en el flujo OAuth).
- El `redirect_uri` debe estar registrado explícitamente en el proveedor.
- El `client_secret` es un secret de backend. Nunca debe estar en el frontend ni en un repositorio.

Datos importantes

- Un atacante **puede leer** el payload de un JWT (está en Base64, no cifrado)
- Un atacante **no puede modificar** el payload sin invalidar la firma
- Si el `JWT_SECRET` se filtra, el atacante **sí puede generar tokens falsos válidos**
- Un JWT expirado con firma válida **debe rechazarse** — el error de parseo no debe ignorarse

HTTPS y TLS

HTTP envía todo en texto plano. Cualquier nodo intermedio (router, ISP, proxy) puede leer y modificar el tráfico. Esto se llama ataque **Man-in-the-Middle (MitM)**. Un atacante en la misma red WiFi puede leer tokens JWT, passwords y modificar respuestas del servidor.

HTTPS = HTTP + TLS. TLS garantiza:

1. **Confidencialidad:** los datos están cifrados.
2. **Integridad:** si los datos se modifican en tránsito, la verificación falla.
3. **Autenticidad:** el certificado TLS prueba que estás hablando con el servidor real.

Certificados TLS: emitidos por una Autoridad Certificante (CA). Let's Encrypt es la CA gratuita y automatizada, estándar para la mayoría de proyectos.

En producción: la terminación TLS generalmente no ocurre en el servidor de la aplicación sino en una capa anterior (Load Balancer, Nginx, Caddy, Cloudflare). El tráfico interno entre esa capa y el servidor puede ir por HTTP.

HTTPS en producción es **no negociable**. En desarrollo local se puede trabajar con HTTP.

Security Headers

Headers HTTP que le indican al navegador qué comportamientos permitir o bloquear. No reemplazan la corrección del código, pero reducen el impacto de errores comunes.

Header	Para qué sirve
<code>Strict-Transport-Security</code>	Fuerza al navegador a usar HTTPS en futuras visitas
<code>Content-Security-Policy</code>	Restringe desde dónde se pueden cargar scripts, estilos, imágenes
<code>X-Content-Type-Options: nosniff</code>	Evita que el navegador adivine tipos de contenido
<code>Referrer-Policy</code>	Controla cuánta información se envía en el header Referer
<code>frame-ancestors</code> (en CSP)	Evita que el sitio sea embebido en iframes no autorizados

Gestión de Secrets

Los secrets son datos sensibles que el sistema necesita pero no deben ser públicos: passwords de base de datos, API keys, JWT secrets, credenciales de OAuth, claves de cifrado.

El problema más común: secrets en el código fuente. Cualquier persona con acceso al repositorio tiene acceso al sistema de producción. Además son difíciles de rotar y servicios como GitHub Secret Scanning los detectan automáticamente.

La solución: variables de entorno. Leer los secrets del entorno en tiempo de ejecución con

```
os.Getenv("JWT_SECRET") .
```

En desarrollo: usar un archivo `.env` que **nunca se commitea al repositorio** (se agrega al `.gitignore`). Se incluye un `.env.example` con las variables necesarias pero sin sus valores reales, para que otros desarrolladores sepan qué deben configurar.

Si accidentalmente commiteaste un secret: no alcanza con borrarlo en un nuevo commit. El historial de git conserva todo. Hay que revocar y regenerar el secret inmediatamente y limpiar el historial con herramientas como BFG Repo Cleaner.

En producción: las variables de entorno se configuran directamente en la plataforma (Railway, AWS, GCP, Kubernetes). Para proyectos complejos se usan herramientas como HashiCorp Vault o AWS Secrets Manager.

Principio de mínimo privilegio: cada componente debe tener acceso solo a los secrets que necesita. Si ese componente se compromete, el daño es limitado.

Testing de seguridad

La seguridad también se prueba. No alcanza con revisar el código: los casos críticos deben tener tests automatizados.

Casos mínimos que deben testearse:

Caso	Resultado esperado
Request sin token a endpoint protegido	401 Unauthorized
Token expirado	401 Unauthorized
Usuario común accede a endpoint admin	403 Forbidden
Usuario A intenta leer recurso de usuario B	404 Not Found o 403 Forbidden
Login con email inexistente	mismo mensaje que password incorrecto
Login con password incorrecto	mismo mensaje que email inexistente
Input malicioso en filtros o búsqueda	no modifica la query ni rompe el servidor
CORS en producción	solo permite orígenes conocidos

Test de ownership (el más importante): verificar que el usuario A no puede leer recursos del usuario B. Este test protege contra una regresión muy común donde alguien refactoriza el endpoint y deja de filtrar por `user_id`.

Priorizar los flujos donde un bug tiene más impacto: acceso a recursos por ID, operaciones administrativas, cambios de estado irreversibles, login y refresh de tokens, creación o edición de usuarios, endpoints con filtros o búsquedas.

Errores comunes

1. Tokens JWT en la URL

Los tokens en URLs aparecen en logs del servidor, historial del navegador y headers Referer. Siempre deben ir en el header `Authorization: Bearer <token>`.

2. Secrets en el código fuente

Ya cubierto en la sección 14. El `.env.example` sin valores reales va al repositorio; el `.env` con los valores reales nunca.

3. Logs con datos sensibles

Los logs pueden quedar almacenados años y tener acceso más amplio que la base de datos. Nunca loguear passwords, tokens JWT completos ni números de tarjeta. Sí loguear el email, el userID y el timestamp de expiración del token.

4. Mensajes de error que revelan información

Si el sistema diferencia "usuario no encontrado" de "password incorrecto", un atacante puede enumerar usuarios válidos. Siempre devolver el mismo mensaje genérico: `"invalid credentials"`.

5. Ignorar la expiración del JWT

Si se parsea el token y se ignora el error de retorno, se pueden procesar tokens expirados. Siempre verificar el error y la propiedad `token.Valid`.

6. JWT con algoritmo "none"

Vulnerabilidad conocida: el atacante modifica el header para usar algoritmo `none` (sin firma), lo que permite falsificar cualquier payload. La defensa es verificar explícitamente que el algoritmo del token sea el esperado (ej: HS256) y rechazar cualquier otro.

7. CORS mal configurado

CORS es una política del navegador que el servidor configura. Permitir `*` (cualquier origen) en

producción anula las protecciones que CORS ofrece. Siempre especificar explícitamente los orígenes permitidos.

8. No validar el input

El cliente nunca debe poder setear campos como el `role` al registrarse. El servidor siempre asigna los valores sensibles, y se deben validar los campos recibidos con restricciones explícitas (required, formato email, longitud mínima de password, etc.).

(9) Docker y Consistencia de Ambientes

El problema que Docker resuelve: "en mi máquina funciona"

El software no corre en el vacío. Depende de todo un entorno de soporte que raramente está bien documentado. Las causas más comunes de inconsistencia entre máquinas son:

- Distintas versiones del lenguaje (Go 1.21 vs Go 1.18 → comportamientos distintos)
- Dependencias del sistema operativo instaladas manualmente y olvidadas
- Diferencias entre sistemas operativos (macOS vs Ubuntu → rutas, case-sensitivity)
- Variables de entorno configuradas en una máquina pero no en otra
- Versiones o configuraciones distintas de MySQL
- Conflictos de puertos

La idea de fondo: cuando ejecutás `go run main.go` no estás corriendo solo tu código. Estás corriendo tu código + el runtime de Go + las librerías del sistema + las variables de entorno + los servicios externos (MySQL, Redis, etc.). Todo eso junto es "el sistema que funciona". El desafío profesional es hacer que ese sistema completo sea reproducible.

Respuesta de la industria: tratar el entorno con el mismo rigor que el código: versionarlo, revisarlo, reproducirlo. Esto se llama **Infrastructure as Code (IaC)**.

Virtualización vs. Contenedores

Máquinas Virtuales (VM)

Emulan hardware completo y corren un sistema operativo entero (con su propio kernel) sobre ese hardware emulado.

- Alto consumo de recursos (cada VM reserva RAM y CPU para su propio SO)
- Inicio lento (segundos a minutos)
- Imágenes de varios GBs
- Aislamiento fuerte a nivel kernel
- Útiles cuando se necesitan sistemas operativos completamente distintos

Contenedores

Un contenedor es un **proceso aislado que comparte el kernel del sistema operativo del host**. No emula hardware, no corre un SO completo. El proceso corre directamente sobre el kernel del host, pero con su propio filesystem, variables de entorno, vista de red y procesos.

- Más livianos (solo las dependencias de la app, no un SO completo)
- Inicio en milisegundos a segundos
- Menor consumo de memoria

- Entorno completamente definido en un archivo

Comparación directa

Dimensión	Máquina Virtual	Contenedor
Sistema operativo	SO completo propio	Comparte kernel del host
Peso típico	GBs	MBs a cientos de MBs
Inicio	Segundos a minutos	Milisegundos a segundos
Aislamiento	A nivel hardware	A nivel proceso
Caso de uso típico	Múltiples SO distintos	Múltiples apps en el mismo SO
Portabilidad	Media	Alta

Nota importante: contenedores y VMs no son excluyentes. En producción es común que los contenedores corran dentro de VMs (el servidor donde corre Docker en AWS o GCP es en sí una VM).

¿Qué es Docker?

Docker es una plataforma que permite **empaquetar, distribuir y ejecutar aplicaciones dentro de contenedores reproducibles**.

Provee:

- Un formato estándar para definir imágenes (el **Dockerfile**)
- Un motor para construir y ejecutar imágenes (`docker build` , `docker run`)
- Un sistema de capas para hacer las imágenes eficientes
- Un registro público de imágenes pre-construidas (**Docker Hub**)
- Una herramienta para orquestar múltiples contenedores (**Docker Compose**)

Qué resuelve Docker

Problema	Solución
"En mi máquina funciona"	El contenedor lleva su propio entorno
Setup manual inconsistente	El entorno se define como código (Dockerfile)
Diferencias entre ambientes	La misma imagen corre igual en dev y producción
Onboarding lento	<code>git clone</code> + <code>docker compose up</code> en lugar de horas de setup
Conflictos de dependencias	Cada contenedor tiene sus propias dependencias aisladas
Testing con infraestructura real	Un contenedor de MySQL que se levanta y destruye en segundos

Qué NO es Docker (confusiones frecuentes para multiple choice)

Confusión	Realidad
Docker = máquina virtual	Un contenedor es un proceso aislado; no emula hardware
Docker = cloud	Docker es una herramienta de empaquetado local
Docker = Kubernetes	Kubernetes orquesta contenedores Docker; son capas distintas
Docker = deploy automático	Docker empaqueta; el pipeline CI/CD se encarga del deploy
Docker = seguridad	Un contenedor no hace seguro al código que corre dentro

Docker no reemplaza: buena arquitectura, tests automáticos, seguridad del código, monitoreo ni backups. Una aplicación mal diseñada sigue siendo una aplicación mal diseñada aunque corra en Docker.

Imagen vs. Contenedor

Esta distinción es fundamental y fuente frecuente de preguntas de examen.

Imagen

Una imagen es una **plantilla inmutable** con todo lo necesario para ejecutar una aplicación. Incluye el sistema de archivos base, el runtime del lenguaje, las dependencias y el binario de la aplicación.

- **No corre.** Es una receta, un molde. No consume CPU mientras nadie la usa.
- **Inmutable:** una vez construida no cambia. Si cambia el código, se construye una nueva imagen.
- **Versionable:** se etiquetan con versiones (`mi-app:1.0` , `mi-app:latest`)
- Se puede subir a un registro (Docker Hub, GitHub Container Registry, Amazon ECR) y compartir.

Contenedor

Un contenedor es una **instancia en ejecución de una imagen**. Cuando ejecutás `docker run` , Docker toma la imagen y arranca un proceso a partir de ella.

- **Efímero:** puede detenerse, reiniciarse o destruirse. Por defecto, si se destruye, todo lo que se escribió dentro se pierde.
- **Tiene estado en runtime:** procesos activos, memoria en uso, archivos que pueden cambiar.
- **Pueden existir múltiples contenedores de la misma imagen**, como múltiples objetos de la misma clase.

La analogía

Concepto	Analogía OOP	Analogía cocina
Imagen	Clase	Receta
Contenedor	Objeto (instancia)	Plato preparado

Por qué los datos se pierden al destruir un contenedor

Una imagen tiene capas de **solo lectura**. Cuando Docker crea un contenedor, agrega encima una **capa de escritura**. Todo lo que la aplicación escribe va a esa capa. Cuando el contenedor se destruye, esa capa desaparece. Por eso los datos persistentes necesitan **volúmenes**.

Filosofía: un proceso por contenedor

El principio: cada contenedor debería tener una única responsabilidad.

En lugar de meter backend + MySQL + nginx en un solo contenedor, se separa en:

- Un contenedor para el backend (Go + Gin)
- Un contenedor para la base de datos (MySQL)
- Un contenedor para el proxy inverso (Nginx), si aplica
- Un contenedor para el cache (Redis), si aplica

¿Por qué?

- **Separación de responsabilidades:** si falla la DB, solo mirás el contenedor de MySQL.
- **Escalabilidad independiente:** podés correr 3 instancias del backend sin tocar la DB.
- **Ciclos de vida independientes:** actualizás MySQL sin tocar el código del backend.
- **Mejor aislamiento:** un crash del backend no afecta al proceso de MySQL.

El Dockerfile

Un Dockerfile es un **archivo declarativo** que describe paso a paso cómo construir una imagen. Es la materialización del principio "infraestructura como código".

Nombre del archivo: `Dockerfile` (sin extensión, D mayúscula), en la raíz del proyecto.

Distinción clave: lo que pasa en el **build** (construcción de la imagen) vs. lo que pasa en el **run** (ejecución del contenedor).

```
FROM golang:1.24      # imagen base
WORKDIR /app          # directorio de trabajo
COPY go.mod go.sum ./ # copiar dependencias primero
RUN go mod download   # descargar dependencias
COPY . .              # copiar el resto del código
RUN go build -o main . # compilar el binario
EXPOSE 8080           # documentar el puerto
CMD ["/main"]         # comando al iniciar el contenedor
```

Instrucciones del Dockerfile

Define desde qué imagen existente se construye. Toda imagen parte de otra. Si se omite el tag, Docker usa `latest`. **En producción es recomendable especificar versión exacta.**

Variantes comunes:

- `golang:1.24` — imagen completa con todas las herramientas (más grande)
- `golang:1.24-alpine` — basada en Alpine Linux, mucho más pequeña, ideal para producción
- `ubuntu:22.04`, `debian:bookworm-slim` — para mayor compatibilidad

WORKDIR — directorio de trabajo

```
WORKDIR /app
```

Establece el directorio de trabajo dentro del contenedor. Todos los comandos siguientes (`COPY`, `RUN`, `CMD`) se ejecutan relativos a este directorio. Si no existe, Docker lo crea. Equivale a `mkdir -p /app && cd /app`.

COPY — copiar archivos

```
COPY go.mod go.sum ./ # copiar archivos específicos
COPY . .              # copiar todo el directorio
```

Copia archivos desde tu máquina (contexto de build) al interior de la imagen. Sintaxis: `COPY <origen> <destino>`.

El orden importa para el cache: copiar primero los archivos de dependencias y descargarlas antes de copiar el código fuente evita re-descargar dependencias cada vez que cambia el código.

RUN — ejecutar durante el build

```
RUN go mod download
RUN go build -o main .
```

Ejecuta un comando durante la **construcción de la imagen**. El resultado queda grabado en una nueva capa. Se usa para descargar dependencias, compilar, instalar herramientas.

Diferencia clave con CMD: `RUN` se ejecuta en el build. `CMD` se ejecuta cuando el contenedor arranca.

EXPOSE — documentar puertos

EXPOSE 8080

Solo documenta qué puerto escucha el proceso. **No abre el puerto automáticamente hacia el host**. Para que sea accesible desde fuera, hay que mapearlo en `docker run -p` o en `docker-compose.yml`.

CMD — comando principal del contenedor

```
CMD [ "./main" ]
```

Define el comando que se ejecuta cuando el contenedor arranca. Es el "proceso principal": si termina, el contenedor se detiene.

Forma array (recomendada) porque evita correr bajo un proceso de shell, haciendo que las señales del sistema (como SIGTERM para shutdown graceful) lleguen directamente al proceso.

CMD vs ENTRYPOINT: CMD es el comando por defecto que se puede sobrescribir al correr el contenedor. ENTRYPOINT define el ejecutable fijo. Para la mayoría de los casos, CMD es suficiente.

Sistema de capas (layering)

Cada instrucción **RUN**, **COPY** y **ADD** genera una nueva **capa** en la imagen. Una imagen es una pila de capas de solo lectura.



Por qué importa el sistema de capas

Caching: cuando reconstruís una imagen, Docker detecta qué capas no cambiaron y las reutiliza. Solo reconstruye a partir de la primera capa que cambió.

Si cambiás solo `main.go`, Docker reutiliza las capas `FROM`, `WORKDIR`, `COPY go.mod go.sum` y `RUN go mod download`. Solo reconstruye las capas que vienen después. Los builds repetidos son mucho más rápidos.

Regla práctica: lo que cambia menos frecuentemente va primero. El código fuente (que cambia seguido) va al final.

```
# BIEN: dependencias primero (cambian poco), código después (cambia seguido)
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN go build -o main .

# MAL: cualquier cambio en el código invalida el cache del go mod download
COPY . .
RUN go mod download
RUN go build -o main .
```

.dockerignore

Archivo que le dice a Docker qué no copiar al construir la imagen. Funciona igual que `.gitignore`.

```
.git
.env
*.md
/tmp
node_modules
```

Reduce el tamaño de la imagen y evita incluir accidentalmente secrets.

Variables de entorno en Docker

La misma imagen debe poder ejecutarse en desarrollo, staging y producción. Lo que cambia es la configuración externa. Si la configuración está dentro de la imagen, necesitas una imagen distinta por ambiente. Si viene de variables de entorno, la misma imagen funciona en todos.

Este principio tiene nombre: **los doce factores del software** (tercer factor: "Almacenar la configuración en el entorno").

Variables típicas en Go + MySQL

```
DB_HOST=mysql
DB_PORT=3306
DB_USER=appuser
DB_PASSWORD=secretpassword
DB_NAME=miapp
JWT_SECRET=supersecret
APP_PORT=8080
```

Archivo .env y .env.example

- `.env` — valores reales para desarrollo local. **Nunca va al repositorio.**
- `.env.example` — variables necesarias sin valores reales. **Sí va al repositorio.** Documenta qué variables se necesitan.

La instrucción ENV en el Dockerfile

```
ENV APP_PORT=8080
```

Solo para valores que **no son secretos** y son iguales en todos los ambientes. **Nunca usar ENV para passwords, tokens o API keys** porque quedan grabados en la imagen y son visibles para cualquiera que la inspeccione.

Redes y comunicación entre contenedores

El problema clave (muy probable en examen)

Si el backend Go corre en un contenedor y MySQL en otro, **no pueden comunicarse usando localhost**. `localhost` dentro del contenedor del backend se refiere al propio contenedor, no a MySQL.

Solución: redes Docker

Docker Compose crea automáticamente una **red privada** para todos los servicios del `docker-compose.yml`. Todos los contenedores de ese Compose están en esa red y se comunican entre sí por el **nombre del servicio**.

Si en el Compose hay un servicio llamado `mysql`, el backend lo alcanza usando `mysql` como hostname:

```
# .env - correcto para dentro de Docker
DB_HOST=mysql

# INCORRECTO - localhost no funciona entre contenedores
DB_HOST=localhost
```

Tipos de redes Docker

- **bridge (default)**: red privada, los contenedores se comunican entre sí y con el host via puertos mapeados. Es la que usa Docker Compose.
- **host**: el contenedor usa directamente la red del host (sin aislamiento). Solo en Linux.
- **none**: sin red. El contenedor está completamente aislado.

Puertos: interno vs. externo

El puerto que escucha la aplicación dentro del contenedor **no es automáticamente accesible desde fuera**. Para acceder desde el host, hay que declarar un mapeo:

```
ports:
  - "8080:8080" # HOST:CONTENEDOR
```

Esto le dice a Docker: "cuando alguien se conecte al puerto 8080 de mi máquina, redirigí la conexión al puerto 8080 de este contenedor".

Ejemplos de mapeos

```
- "8080:8080" # caso más común
- "3001:3000" # si el 3000 ya está ocupado en el host
- "3307:3306" # MySQL, si ya tenés MySQL local en el 3306
```

Cuándo NO mapear puertos

Si un servicio solo necesita ser accesible por otros contenedores (no desde el host), no hace falta mapear. El backend Go puede hablar con MySQL dentro de la red Docker sin que MySQL tenga

puertos mapeados al host. Mapear el puerto de MySQL al host es útil solo para conectarte desde herramientas como DBeaver o TablePlus.

Persistencia y volúmenes

El problema

Los contenedores son **efímeros**. Al destruir un contenedor (`docker compose down`), la capa de escritura desaparece. Para una base de datos, esto significa perder todos los datos.

Solución: volúmenes

Un volumen es un directorio gestionado por Docker que existe en el **host** y se monta dentro del contenedor. Cuando el contenedor escribe en esa ruta, en realidad está escribiendo en el host.

Tipos de montajes

Volumen nombrado (el más común en Docker Compose):

```
volumes:
  - mysql_data:/var/lib/mysql
```

Docker gestiona dónde se almacena `mysql_data` en el host. El dato persiste aunque el contenedor se destruya.

Bind mount (montar un directorio del host directamente):

```
volumes:
  - ./src:/app/src
```

Útil en desarrollo para que los cambios en el código se reflejen dentro del contenedor sin reconstruir la imagen.

Comportamiento de `docker compose down`

```
docker compose down      # detiene contenedores, CONSERVA los volúmenes
docker compose down -v   # detiene contenedores Y BORRA los volúmenes
```

MySQL almacena sus datos en `/var/lib/mysql`. Siempre declarar un volumen para ese directorio:

```
services:
  mysql:
    image: mysql:8.0
    volumes:
      - mysql_data:/var/lib/mysql

volumes:
  mysql_data:
```

Docker Compose

Docker Compose permite definir y ejecutar **múltiples contenedores como un sistema unificado** usando un único archivo declarativo: `docker-compose.yml`.


```
docker compose up    # levantar todo el sistema
docker compose down  # detener todo
```

Beneficios

- Un único archivo declarativo, versionado junto al código
- Levantar todo con un comando
- Redes automáticas entre servicios
- Variables compartidas desde el archivo `.env`
- Manejo de dependencias (`depends_on`)

Comandos esenciales de Docker Compose

Comando	Descripción
<code>docker compose up</code>	Levanta todos los servicios en primer plano
<code>docker compose up -d</code>	Levanta en segundo plano (detached)
<code>docker compose down</code>	Detiene y elimina contenedores
<code>docker compose down -v</code>	Detiene, elimina contenedores y volúmenes
<code>docker compose logs</code>	Muestra logs de todos los servicios
<code>docker compose logs -f backend</code>	Sigue los logs en tiempo real
<code>docker compose build</code>	Reconstruye las imágenes sin levantar
<code>docker compose ps</code>	Muestra el estado de los contenedores
<code>docker compose exec backend sh</code>	Abre una shell dentro del contenedor
<code>docker compose restart backend</code>	Reinicia solo ese servicio

`depends_on` y su limitación importante

`depends_on: mysql` hace que Compose arranque MySQL antes que el backend. **Pero solo espera a que el contenedor esté corriendo, no a que MySQL esté listo para aceptar conexiones.** MySQL puede tardar 5-10 segundos en inicializarse. Si el backend intenta conectarse en ese instante, falla. La solución más simple es implementar **retry en el código Go** al inicializar la conexión GORM.

Ambientes: desarrollo, testing y producción

Los ambientes en una empresa real

Ambiente	Propósito
Local (dev)	Desarrollo activo. Datos de prueba. Configuración laxa.
Testing/QA	Verificación de features. Datos controlados. Sin tráfico real.
Staging	Réplica de producción. Datos anonimizados. Validación final.
Producción	Usuarios reales. Datos reales. Máxima disponibilidad.

La idea central con Docker

La misma imagen debe poder correr en todos los ambientes. Lo que cambia es la configuración externa.

Una práctica incorrecta es tener un Dockerfile para dev y otro para producción. La imagen es el artefacto que se valida en testing y se despliega en producción. Si la imagen cambia entre

ambientes, perdés la garantía de que lo que testeaste es lo que desplegás.

Lo que sí puede variar entre ambientes: variables de entorno, infraestructura externa (distintas bases de datos), archivos Compose.

Onboarding con Docker

```
git clone https://github.com/org/proyecto.git
cd proyecto
cp .env.example .env      # completar los valores
docker compose up
```

Eso es todo. En minutos el sistema completo está corriendo, independientemente del SO del desarrollador.

Docker y testing

Docker mejora el testing porque:

- **Entornos reproducibles:** el entorno de testing es exactamente el mismo en cada máquina y en CI
- **Base de datos aislada para testing:** se levanta MySQL específicamente para tests, sin contaminar desarrollo
- **Misma versión de dependencias:** si los tests pasan con MySQL 8.0.32, pasan con MySQL 8.0.32 en todos lados
- **Setup automático:** los tests de CI no requieren configuración manual

Flujo típico en CI/CD

1. `git clone` del código
2. `docker compose up -d` (levantar backend + MySQL)
3. Esperar a que los servicios estén healthy
4. Correr tests: `go test ./...`
5. `docker compose down -v` (limpiar todo)
6. Si todos los tests pasaron → deployar

Logs y observabilidad básica

Los sistemas modernos asumen que las aplicaciones en contenedores escriben logs a **stdout** y **stderr**, no a archivos en disco.

¿Por qué? Docker (y Kubernetes) captura stdout/stderr de todos los contenedores y los centraliza automáticamente. Si la aplicación escribe a un archivo dentro del contenedor, esos logs:

- Se pierden cuando el contenedor se destruye
- Son difíciles de agregar de múltiples instancias
- Requieren acceso al filesystem del contenedor para leerlos

El logger estándar de Go ya escribe a stdout por defecto.

Buenas prácticas de logging:

- Nunca escribir datos sensibles en logs (passwords, tokens, números de tarjeta)

- Usar logs estructurados (formato JSON) en producción para facilitar el análisis automático
- Nivel de log apropiado: DEBUG en desarrollo, INFO/WARN/ERROR en producción

Buenas prácticas generales

Mantener imágenes pequeñas: usar imágenes base minimalistas (`alpine`), multi-stage builds, `.dockerignore`.

No hardcodear configuración: la configuración viene de variables de entorno, nunca está incrustada en la imagen.

Un servicio por contenedor: principio de responsabilidad única aplicado a contenedores.

No guardar estado dentro del contenedor: el filesystem es efímero. Todo estado persistente va en volúmenes.

Versionar Dockerfile y Compose: son parte del código del proyecto, van en el repositorio y pasan por code review.

No correr como root en producción: crear un usuario sin privilegios y ejecutar la aplicación como ese usuario. Si hay una vulnerabilidad que permite ejecución de código arbitrario, que corra como usuario limitado reduce el daño.

Usar versión explícita en FROM: `latest` puede cambiar con actualizaciones incompatibles. Siempre especificar versión exacta para builds reproducibles.

Errores frecuentes (muy relevantes para multiple choice)

Error	Por qué falla	Corrección
<code>DB_HOST=localhost</code> entre contenedores	<code>localhost</code> en el contenedor del backend se refiere a sí mismo	Usar el nombre del servicio: <code>DB_HOST=mysql</code>
Hardcodear passwords en el Compose	El secret queda en el historial de git para siempre	Usar variables del <code>.env</code> : <code>DB_PASSWORD=\${DB_PASSWORD}</code>
No declarar volumen para MySQL	Al hacer <code>docker compose down</code> se pierden todos los datos	Siempre declarar volumen para <code>/var/lib/mysql</code>
Múltiples responsabilidades en un contenedor	Se pierde aislamiento, escalabilidad y ciclos de vida independientes	Un servicio por contenedor
Crear que cambiar el código actualiza el contenedor	Una imagen es estática; el cambio no se refleja hasta reconstruir	<code>docker compose up --build</code>
<code>COPY . .</code> sin <code>.dockerignore</code>	Archivos sensibles (<code>.env</code> , <code>.git</code>) entran en la imagen	Definir siempre un <code>.dockerignore</code>
Ignorar la limitación de <code>depends_on</code>	Solo espera que el contenedor corra, no que MySQL esté listo	Implementar retry en la conexión GORM
Usar <code>ENV</code> para secrets en el Dockerfile	Quedan grabados en la imagen, visibles para cualquiera	Pasar secrets via variables de entorno externas

Resumen de conceptos clave

Concepto	Definición
Contenedor	Proceso aislado que comparte el kernel del host; tiene su propio filesystem, red y variables de entorno
Imagen	Plantilla inmutable con todo lo necesario para ejecutar un contenedor; se construye desde un Dockerfile

Concepto	Definición
Dockerfile	Archivo declarativo que describe cómo construir una imagen, instrucción por instrucción
Docker Compose	Herramienta para definir y correr múltiples contenedores como un sistema unificado
Volumen	Mecanismo para persistir datos fuera del contenedor, en el host
Red bridge	Red privada que Docker Compose crea automáticamente; los contenedores se comunican por nombre de servicio
Capa (layer)	Unidad de una imagen generada por cada instrucción del Dockerfile; se cachea entre builds
.dockerignore	Lista lo que no debe incluirse al copiar al contexto de build
depends_on	Directiva de Compose que ordena el arranque de los servicios (solo el arranque, no la disponibilidad)
Multi-stage build	Técnica para separar la etapa de compilación de la de ejecución, reduciendo el tamaño de la imagen final
Infrastructure as Code	El entorno de ejecución se define en archivos de texto versionables junto al código